

Anamorphic Channels in Protocol Randomness

Nickleby Simmonds



4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2026

Abstract

In environments where online communication is monitored or censored, covert channels provide a means for parties to exchange hidden information within ordinary encrypted traffic while remaining indistinguishable to external observers. Anamorphic encryption provides a framework for achieving this goal, allowing ciphertexts to carry both an overt message and a covert message recoverable only by parties possessing a special secret key. This dissertation investigates how anamorphic mechanisms can be integrated into widely deployed cryptographic systems while preserving their normal behaviour and security guarantees.

We design two new constructions that embed covert channels into the randomness of existing protocols. The first scheme extends the authenticated encryption scheme AES-GCM by encoding covert data within the initialisation vector using a lightweight XOR-based technique. This approach enables substantially higher bandwidth than rejection sampling methods. The second scheme introduces a covert channel into the Signal messaging protocol by embedding hidden information into the ephemeral ratchet keys during the asymmetric ratchet steps of the Double Ratchet algorithm.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(Nickleby Simmonds)

Acknowledgements

Many thanks, firstly, to my supervisor, Michele Ciampi, for his insight and support over the past year.

To Ria, my flatmates, and all the friends who have made the last four years a movie.

Finally, to my parents and sister, for all the little things I could never live without.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Model	2
1.3	Existing Work	3
1.4	Contributions	4
2	Anamorphic AES-GCM	6
2.1	Preliminaries	6
2.2	Standard AES-GCM	7
2.3	Covert-IV Scheme	8
2.4	Syntax	9
2.5	Security Definition	10
2.5.1	Indistinguishability	11
2.5.2	Preservation of AES-GCM’s Security Guarantees	11
2.5.3	Confidentiality of the Covert Message	12
2.6	Security Proofs	12
2.7	IV Reuse	15
2.7.1	PRF Collisions	16
2.7.2	Rekeying Policy	16
2.8	Robustness	16
2.9	Attacks	18
2.9.1	Replay	18
2.9.2	Re-encryption	18
2.9.3	Message Dropping	18
2.9.4	Timing attacks	19
2.10	Discussion	20
3	Anamorphic Signal	22
3.1	Preliminaries	22
3.2	Signal Protocol	23
3.2.1	X3DH Handshake	23
3.2.2	Symmetric Ratchet	24
3.2.3	Asymmetric Ratchet	24
3.3	Covert-Keyspace Scheme	25
3.4	Syntax	26

3.5	Security Definition	27
3.5.1	Indistinguishability	28
3.5.2	Preservation of Signal's Security Guarantees	29
3.5.3	Confidentiality of the Covert Message	29
3.6	Security Proofs	30
3.7	Repetition of Keys	32
3.7.1	Rekeying Policy	33
3.8	Computational Feasibility	34
3.9	Robustness	35
3.9.1	Incremental Authentication	35
3.10	Attacks	38
3.10.1	Safety under Compromised Assumptions	38
3.10.2	Potential Attacks	38
3.11	Discussion	39
4	Conclusions	40
	Bibliography	41
A	Complete Proofs	44
A.1	Preservation of Signal Guarantees	44
A.2	Covert-Keyspace Confidentiality	44
B	Masked Covert-Keyspace Construction	46
B.1	Comparison with Rejection Sampling	46
B.2	Security of the Masked Covert-Keyspace Scheme	47

Chapter 1

Introduction

1.1 Motivation

Cryptography underpins modern claims to privacy in an increasingly digital world. The growing dependence on electronic communication has led to vast amounts of our information and conversations being transmitted across public channels that are susceptible to interception. This challenge motivated the development of public-key encryption, an algorithm that enables two parties to communicate securely using a pair of mathematically related keys (Diffie and Hellman, 1976). This principle provides a foundation for widely deployed protocols such as TLS (Rescorla, 2018), IPsec (Kent and Seo, 2005), and SSH (Ylönen and Lonvick, 2006). Although these systems provide strong security guarantees under standard cryptographic assumptions, they do not address scenarios in which users may be compelled to reveal secret information.

Historical evidence demonstrates that large-scale surveillance has already been realised in practice. For example, the East German Stasi relied on extensive human intelligence networks, with over 1% of the population serving as informers by the 1980s (Lichter et al., 2021). Modern digital infrastructure has significantly expanded these capabilities, enabling the interception of communications on a far greater scale (Villasenor, 2011).

Surveillance may extend beyond passive observation. It can also involve legal or institutional pressure to obtain access to sensitive information. Currently, some jurisdictions permit state-controlled network filtering and mandate that companies provide technical assistance to decrypt user data for security services (Freedom House, 2021; Polyakova and Meserole, 2019). Similar trends are appearing in democratic settings. In 2025, the UK Home Office reportedly issued a Technical Capability Notice under the Investigatory Powers Act requiring Apple to provide access to encrypted iCloud backups, which prompted the company to withdraw the feature for UK users (Apple Inc., 2025).

This raises an interesting question: Can cryptographic systems provide meaningful privacy guarantees even when adversaries can legally compel users to comply with their demands? In particular, can users communicate hidden information in a way that remains undetectable, even if adversaries observe all communications and possess the encryption randomness and keys?

A natural approach is to conceal information by making its existence undetectable. This dissertation addresses these questions through the lens of *anamorphic encryption*, a paradigm that enables covert communication embedded within standard ciphertexts.

1.2 Model

Persiano, Phan, and Yung observe that most practical cryptographic schemes rely on two underlying assumptions (Persiano et al., 2022):

Sender-Freedom Assumption. The sender (Alice) is free to choose the message to be sent.

Receiver-Privacy Assumption. The receiver’s (Bob’s) secret key remains uncompromised.

These assumptions are reasonable in classical threat models, where adversaries are limited to traditional attacks on communication channels. However, they do not capture scenarios in which adversaries can legally coerce participants into breaking these assumptions. To model such settings, we adopt a strengthened adversarial model inspired by Persiano et al.

Compromised-Sender Assumption. The adversary (Eve) may coerce Alice as follows:

- Eve may choose the plaintext message and require Alice to encrypt it.
- Eve may choose the recipient by selecting the long-term public key under which the ciphertext is produced.
- After encryption, Eve may request the randomness used to produce the ciphertext and verify that the computation was performed correctly.

Intuitively, we allow Eve to force Alice to send an adversarially-chosen message to a recipient of Eve’s choice. This assumption is motivated by dynamics observed in real authoritarian contexts (Lidman, 2026). Importantly, Eve does not control the randomness used during encryption. Allowing the adversary to bias or choose this randomness would amount to the sender abdicating full control over the encryption process. This effectively reduces the model to impersonation rather than coercion. Instead, Alice samples the randomness independently, but may be required to reveal it after the fact, enabling the adversary to verify correctness.

Compromised-Receiver Assumption. Eve may coerce Bob as follows:

- Eve has full control over the communication channel, and may eavesdrop, replay, delay, modify, inject, or drop messages.
- Eve may request Bob’s secret key and verify its correctness.

Thus, we consider an adversary with full network control, as in the Dolev-Yao model (Dolev and Yao, 1981), augmented with the ability to compel disclosure of secret keys.

1.3 Existing Work

An anamorphic encryption scheme allows two parties who share an additional secret, referred to as a *double key*, to embed covert messages within ciphertexts of an established public-key encryption scheme. In this setting, the public and secret keys operate as usual, encrypting and decrypting normal messages. The double key, however, enables the sender to encode hidden data in the message. Thus, a ciphertext can carry two layers of information: an overt message, recoverable by standard decryption, and a covert message, accessible only to a receiver who knows the double key. Crucially, ciphertexts containing hidden data remain computationally indistinguishable from ordinary ciphertexts, preventing even a powerful adversary from detecting the covert channel.

This paradigm was first introduced by Persiano et al., who presented two initial anamorphic constructions. The first relies on *rejection sampling* to encode covert information into the randomness of the ciphertext. As this scheme is relevant to our contributions, we include an informal recollection. The sender repeatedly generates candidate ciphertexts of the normal message by varying the randomness used in probabilistic encryption. This continues until one successfully encodes a hidden bit when a pseudorandom function condition, keyed by the double key, is applied to it. The receiver can then recover the covert bit by computing the same pseudorandom function on the received ciphertext. An observer sees ciphertexts indistinguishable from ordinary encryptions. The second construction adapts the Naor-Yung paradigm and embeds covert messages of the same size as the overt plaintext. This approach involves a modified non-interactive zero-knowledge proof that two ciphertexts encrypt the same plaintext without revealing the plaintext.

An important limitation of these approaches is the number of covert bits per communication, which we refer to as the covert channel’s bandwidth. Rejection sampling constructions embed a small number of bits per ciphertext, as the number of trials needed grows exponentially with each additional bit. Although constructions based on the Naor-Yung paradigm allow much larger covert messages, they require additional data to be transmitted. As a result, achieving high-capacity covert channels without introducing detectable structural changes is still a significant practical challenge. We contribute a scheme that provides a covert capacity exceeding that achievable by rejection sampling, while requiring no additional data beyond the underlying protocol.

Following this initial work, Banfi et al. (Banfi et al., 2024) identified and addressed several limitations of the original paper. First, the Naor-Yung construction requires that the double key be generated simultaneously with the public and secret key pair, making it impossible to add a covert channel to an already-deployed public key. In contrast, both schemes presented in this dissertation allow parties to initiate covert communication at any time after establishing a double key. The other limitation concerns robustness: in the original constructions, a receiver attempting to decode a ciphertext without a hidden message would obtain meaningless output. Banfi et al. formalised a robustness requirement that an anamorphic scheme should provide an explicit indication when no covert message is present. To satisfy this, most existing solutions require synchronised counters or selective randomness recoverability. One of our schemes

goes further, achieving robustness without such assumptions and additionally providing protection against message-dropping attacks that would otherwise disrupt anamorphic communication.

Several papers expand the theory and constructions of anamorphic encryption. Catalano, Giunta, and Migliaro (Catalano et al., 2024) showed that hybrid encryption, IBE-to-CCA transforms, and even certain homomorphic encryption schemes can be naturally extended to support covert messages. Furthermore, they proposed splitting the double key into separate encryption and decryption components to enable multiple independent covert channels. Despite these advances, current research on applying anamorphic ideas beyond public-key encryption remains limited. This gap inspired this work to explore how anamorphism can be adapted to symmetric encryption and a modern messaging protocol.

More recent work has investigated the limits of anamorphic encryption. Dodis and Goldin (Dodis and Goldin, 2025) introduced the notion of anamorphic-resistant encryption, referring to encryption schemes that provably admit no hidden communication channel, and Carnemolla et al. (Carnemolla et al., 2025) provided concrete constructions of such schemes using lossy-function techniques. These results demonstrate that covert channels cannot be trivially added to all encryption schemes, and that practical constructions require high-entropy ciphertexts or structural assumptions. This motivates our approach: we target universally adopted protocols with suitable properties that enable continuous embedding of hidden data while maintaining indistinguishability.

1.4 Contributions

Persiano et al. argued that practical anamorphic schemes should ideally extend existing, widely deployed cryptographic systems rather than introduce entirely new primitives. Bespoke encryption schemes designed for covert messaging are more likely to attract scrutiny or be banned outright. Our work develops that idea into a broader paradigm: *anamorphic channels in protocol randomness*. The key insight is that many protocols expose public fields that are required only to be indistinguishable from uniformly random to an external observer. By encoding covert information into these values while preserving the protocol’s visible behaviour, we obtain hidden communication channels that are difficult to detect. Crucially, this approach decouples the covert channel from the underlying key generation process, meaning parties that share a double key can initiate covert communication at any time without modifying existing public keys or reinitialising the protocol.

Our first construction exploits the randomness of initialisation vectors (IVs), which are non-secret values that ensure identical plaintexts encrypt to different ciphertexts. By replacing this randomness with a one-time pad implemented using a pseudorandom function keyed by a double key, it becomes possible to embed covert data directly while preserving its distribution. Concretely, Alice computes a pseudorandom mask from the double key and a shared counter, and XORs this mask with the covert message to obtain an IV. The IV is used in encryption and transmitted in the clear to enable decryption, creating an ongoing hidden channel without altering the message format

or transmitting additional bits. In Chapter 2, we construct an anamorphic extension of AES-GCM using this approach. This scheme achieves higher covert capacity than prior rejection-sampling constructions, preserves the standard ciphertext and authentication tag structure, and supports continuous covert communication. To our knowledge, this is the first anamorphic construction for a symmetric encryption protocol.

The second construction exploits entropy in public keys. Because public keys are drawn uniformly from a large space, they can serve as carriers for covert information via rejection sampling. Specifically, Alice repeatedly samples candidate keys until one encodes the desired covert message when a pseudorandom function is applied to it. Adapting this idea to the Signal protocol in Chapter 3, we embed covert messages into ephemeral public keys which are transmitted in the clear whenever the direction of communication changes. This construction integrates naturally with the Double Ratchet, preserves forward and post-compromise security, and allows ongoing covert communication. To our knowledge, this is the first anamorphic construction compatible with Signal’s ratchet architecture.

Across both constructions, we formalise security and robustness using adapted definitions of anamorphic encryption. We define indistinguishability and confidentiality games for both constructions, and prove that the anamorphic extensions preserve the security guarantees of the underlying protocols. In addition, we present new mechanisms for robust decoding that allow receivers to detect the presence of covert data and any attempted modifications. We consider an expanded set of adversarial attacks arising from the compromised assumptions, and analyse the resilience of our constructions against these stronger adversarial capabilities.

Finally, we implement both constructions and evaluate them against the underlying protocols, quantifying the trade-offs between covert capacity and computational overhead. These results demonstrate that the schemes are not only theoretically sound but also practically feasible.

Chapter 2

Anamorphic AES-GCM

2.1 Preliminaries

Notation

Throughout this work, we will use the following notation.

$\text{Enc}_k(\cdot)$ denotes symmetric encryption under key k , $\text{Dec}_k(\cdot)$ denotes decryption.

For asymmetric primitives, (pk, sk) denotes a public/private key pair. Random sampling is denoted by $x \xleftarrow{\$} S$, meaning that x is sampled uniformly at random from the set S .

We distinguish between two classes of adversaries. A *standard adversary* operates under conventional cryptographic assumptions. A *compromised adversary* models a stronger opponent that may additionally act in accordance with the compromised-sender and compromised-receiver assumptions. If a scheme is secure against a compromised adversary, it is also secure against a standard adversary. We model all adversaries as probabilistic polynomial-time (PPT) adversaries.

We analyse computational security asymptotically in the security parameter λ . The notation $\text{poly}(\cdot)$ denotes a polynomial function, and $\text{negl}(\cdot)$ denotes a negligible function, meaning it decreases faster than the inverse of any polynomial.

Finally, we distinguish between *overt* and *covert* messages. The overt message m denotes the plaintext processed by the standard encryption protocol, while the covert message m_c denotes hidden information embedded within the protocol outputs for secret communication between parties.

Pseudorandom Functions, Key Derivation, and Message Authentication

Our constructions make use of standard cryptographic primitives: pseudorandom functions (PRFs), key derivation functions (KDFs), and message authentication codes (MACs).

A *pseudorandom function* is a keyed function $\text{PRF}_k(\cdot)$ whose output is computationally indistinguishable from that of a truly random function to any PPT adversary that does

not know the key k (Katz and Lindell, 2014).

A *key derivation function* $\text{KDF}(\cdot)$ expands a secret input into one or more cryptographically independent subkeys.

A *message authentication code* is a keyed function $\text{MAC}_k(\cdot)$ that produces a tag that allows a receiver to verify the integrity and authenticity of a message. We assume that all MACs used in this work are existentially unforgeable under chosen-message attacks (EUF-CMA). Informally, this means that no PPT adversary can produce a valid tag on any new message except with negligible probability, even after obtaining tags on messages of its choice.

Throughout this work, we assume the existence of secure PRFs, KDFs, and MACs. Their security guarantees hold against PPT adversaries with negligible advantage in the security parameter λ .

Security Games and Advantage

We adopt the game-based approach to concrete security, as put forward by Bellare et al. (Bellare et al., 1994), between a challenger and a PPT adversary $\mathcal{A}$. Each game begins with an *init* phase, and provides $\mathcal{A}$ with *oracles* to interact with the system. After interacting, $\mathcal{A}$ outputs a guess bit $b' \in \{0, 1\}$ representing the game it believes it is interacting with.

Let $\mathcal{G}_0$ be the “ideal” security game and $\mathcal{G}_1$ the “real” security game. Then $\Pr[\mathcal{G}_0^{\mathcal{A}}(\lambda) = 1]$ is the probability $\mathcal{A}$ outputs $b' = 1$ in the ideal game, and $\Pr[\mathcal{G}_1^{\mathcal{A}}(\lambda) = 1]$ is the probability $\mathcal{A}$ outputs $b' = 1$ in the real game.

2.2 Standard AES-GCM

AES-GCM is a symmetric authenticated-encryption mode that combines AES in counter mode with a polynomial-based (Galois) message authentication code. This scheme is used as the default encryption scheme in modern security protocols such as TLS, SSH, and Signal, due to its ability to exploit hardware-level parallelism for high-speed encryption and decryption (Dworkin, 2007).

To encrypt a plaintext, we first derive a series of blocks from an initialisation vector IV. Each of these blocks is encrypted under the symmetric key k to produce keystream blocks. These are XORed with blocks of the plaintext to create ciphertext blocks, which, when concatenated, form the complete ciphertext C. To ensure integrity, a 128-bit authentication tag T is computed by applying a keyed hash function over the ciphertext, IV, and any associated data (AAD). The hash key is derived from k , ensuring that the correct tag can be recreated only by recipients with the symmetric key (Katz and Lindell, 2014).

The sender typically transmits $\langle \text{IV}, \text{C}, \text{T} \rangle$ although, as we will see in Section 2.10, this is not always true in real-world implementations. Successful verification of T allows the recipient to confirm, with overwhelming probability, that the ciphertext, AAD, and IV have not been altered.

Against standard adversaries, AES-GCM provides AEAD security (McGrew and Viega, 2007). Informally, this security notion requires that an adversary learns no information about the underlying plaintext from the ciphertext (IND-CPA), and cannot forge a valid ciphertext–associated data pair that would be accepted as authentic without access to the secret key (INT-CTXT).

IV requirements

A critical requirement of GCM is that, for a fixed key, every IV must be distinct. If two messages are encrypted with the same (k, IV) pair, security is catastrophically broken: the keystream repeats, and the authenticity guarantee fails.

One method of generating IVs is random sampling. NIST suggests that when IVs are chosen uniformly at random the probability of IV reuse should be $\leq 2^{-32}$ and that the total number of encryptions under a single key should be limited to 2^{32} messages; rekeying is recommended before this bound is exceeded (Kampanakis et al., 2024). Typically, IVs of 96-bits are used; we will denote the length of this field by $|IV|$.

2.3 Covert-IV Scheme

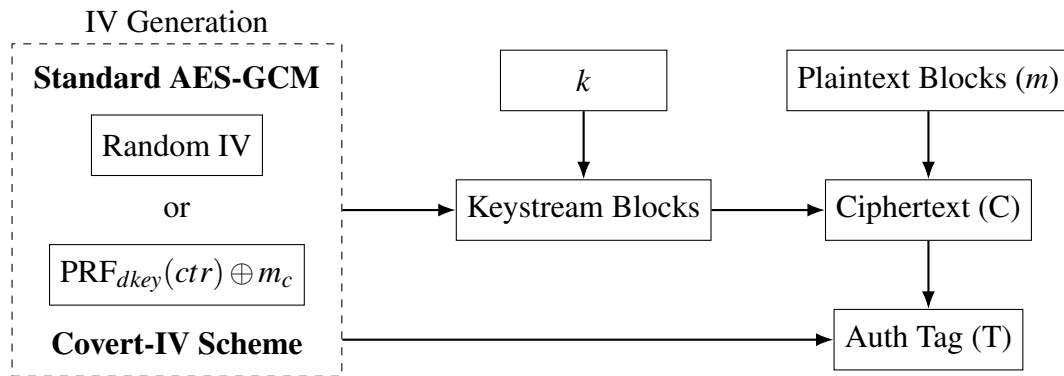


Figure 2.1: Abstracted visualisation of encryption under standard AES-GCM and the covert-IV scheme. Encryption outputs $\langle IV, C, T \rangle$.

We propose an extension of the AES-GCM protocol described in the previous section. The covert message shall be encoded within the randomly generated IVs; we will refer to this scheme as the covert-IV scheme.

Before initiating communication, Alice and Bob pre-share a secret key called the double key ($dkey$). They also individually initialise a shared counter (ctr) that increments after each communication. These values are assumed to be established in person or before the adversary (Eve) has the legal ability to compel the disclosure of secret keys.

When Alice needs to send a covert message to Bob, she generates a key pair (pk, sk) and publishes pk . Then, they perform a handshake protocol to establish a secure channel and derive a shared symmetric session key k . To send a message, instead of using a randomly generated bit-string, Alice applies a PRF keyed with $dkey$ to the ctr , as

shown in Figure 2.1, and produces an output that is computationally indistinguishable from random. This value is XORed with her covert message m_c and is used as the IV during encryption. Using k and the chosen IV, Alice executes the standard AES-GCM encryption algorithm to encrypt the overt message m and obtains a ciphertext C and an authentication tag T . Alice transmits the tuple $\langle IV, C, T \rangle$.

Upon receiving the broadcasted tuple, Bob decrypts C , using k and the transmitted IV to recover m . Independently, Bob also computes $\text{PRF}_{dkey}(ctr)$ and XORs the result with IV to recover m_c .

If Eve compels Bob to disclose his private key, Bob surrenders sk , which allows Eve to decrypt the handshake, obtain k , and decrypt m . Critically, she gains no information about m_c embedded in IV, because Eve does not possess $dkey$. Equally importantly, Eve does not know and cannot prove that a double key exists or that any covert communication is taking place.

To evaluate the security and correctness of this construction rigorously, we now define the formal syntax of the Covert-IV scheme.

2.4 Syntax

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ denote the standard AES-GCM authenticated encryption scheme with random IVs. For a security parameter λ , key generation, encryption, and decryption are defined as

$$k \xleftarrow{\$} \text{Gen}(1^\lambda), \quad \langle IV, C, T \rangle \leftarrow \text{Enc}(k, m), \quad m \leftarrow \text{Dec}(k, \langle IV, C, T \rangle).$$

Note that we distinguish between $\text{Enc}(k, m)$ and $\text{Enc}_k(IV, m)$. $\text{Enc}(k, m)$ encapsulates randomly sampling an IV and then computing $(C, T) := \text{Enc}_k(IV, m)$. Similarly, $\text{Dec}(k, \langle IV, C, T \rangle)$ computes $m := \text{Dec}_k(\langle IV, C, T \rangle)$.

Definition 2.4.1 (Covert-IV Scheme). Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^{|IV|}$ be a secure pseudorandom function, where λ is the security parameter and $n \geq 32$ is the length of a counter input. $dkey$ is shared out-of-band and ctr is initialised at 0. The scheme Π' extends Π as follows:

- $\text{aGen}(1^\lambda)$: Returns a symmetric key k and a double key $dkey$.
- $\text{aEnc}(k, dkey, ctr, m, m_c)$: Computes $IV := \text{PRF}_{dkey}(ctr) \oplus m_c$, where $m_c \in \{0, 1\}^{|IV|}$. The ciphertext and tag pair is then encrypted as in Π : $(C, T) := \text{Enc}_k(IV, m)$. The output is $\langle IV, C, T \rangle$.
- $\text{aDec}(k, dkey, ctr, \langle IV, C, T \rangle)$: Decrypts the ciphertext to obtain $m := \text{Dec}_k(\langle IV, C, T \rangle)$ and recovers the covert message as $m_c := \text{PRF}_{dkey}(ctr) \oplus IV$.

The covert-IV scheme Π' can operate in four distinct modes while still providing correctness:

- $\Pi'_{\text{Enc, Dec}}$: Encryption is extended, decryption is standard. Correctness: $\text{Dec}(k, \text{Enc}(k, m)) = m$.

- $\Pi'_{\text{aEnc,Dec}}$: IVs embed a covert message, but the receiver recovers only the overt message m . Correctness: $\text{Dec}(k, \text{aEnc}(k, dkey, ctr, m, mc)) = m$.
- $\Pi'_{\text{Enc,aDec}}$: IVs contain no covert data; any reconstructed covert message is uniformly random (m_c^*). Receiver recovers m . Correctness: $\text{aDec}(k, dkey, ctr, \text{Enc}(k, m)) = (m, m_c^*)$.
- $\Pi'_{\text{aEnc,aDec}}$: Full anamorphic exchange. Correctness: $\text{aDec}(k, dkey, ctr, \text{aEnc}(k, dkey, ctr, m, mc)) = (m, m_c)$.

In the intermediate modes an adversary gains no additional advantage therefore their security and robustness follow directly from the guarantees of Π and the full covert-IV scheme $\Pi'_{\text{aEnc,aDec}}$. Henceforth, we focus our analysis on $\Pi'_{\text{aEnc,aDec}}$ and denote it simply as Π' .

2.5 Security Definition

To simplify the presentation and analysis of security claims, we make the following additional assumptions.

- **Asymmetric Abstraction.** The mechanism by which Alice and Bob establish a symmetric session key k is abstracted away. In practice, this key is derived via a standard handshake protocol using public-private key pairs (pk, sk) held by each party. Since the details of this process are irrelevant to the security of our construction, we model it as part of the key generation.
- **Compromised Advantage.** In the following games, we formalise the compromised-sender assumption through oracle access in our security games. The adversary selects plaintexts via oracle queries, and receives ciphertexts of the form $\langle \text{IV}, C, T \rangle$. Since AES-GCM explicitly includes the IV in the ciphertext, this reveals the randomness used during encryption, allowing the adversary to verify correct execution. Similarly, we do not limit the adversary's ability to choose the recipient of the message.

Under the compromised-receiver assumption, the adversary can compel disclosure of Bob's long-term secret key, which in a real deployment would allow recovery of the session key k by decrypting the handshake transcript. Therefore, in our security games, we provide the adversary with k directly.

- **Non-Repeating IVs.** We assume that the initialisation vectors generated by the covert-IV scheme do not repeat under the same session key k . This assumption is required for the standard security guarantees of AES-GCM to hold. The probability of IV reuse is analysed explicitly in Section 2.7.
- **Counter Synchronisation.** We assume that Alice and Bob maintain a consistent view of the counter ctr . Handling desynchronisation is discussed in Section 2.9.3.

2.5.1 Indistinguishability

We define two distinguishing games to capture the ability of an adversary $\mathcal{A}$ to distinguish between executions of Π and Π' . $\mathcal{A}$ can issue up to $\text{poly}(\lambda)$ oracle queries with adaptive overt messages m and covert messages $m_c \in \{0, 1\}^{|\text{IV}|}$, after which the adversary outputs a bit b' indicating which game it believes it is interacting with.

Both games provide the adversary with the compromised assumptions and the same oracle:

$O(m, m_c)$: encrypts plaintext m , optionally embeds m_c , and returns $\langle \text{IV}, \text{C}, \text{T} \rangle$.

Game $\text{DistGame}_0^{\Pi, \mathcal{A}}(\lambda)$ (Standard AES-GCM) Init: 1. $k \xleftarrow{\\$} \text{Gen}(1^\lambda)$ 2. Give k to adversary Oracle $O(m, m_c)$: 1. Return $\langle \text{IV}, \text{C}, \text{T} \rangle := \text{Enc}(k, m)$ Output: Adversary outputs guess b'	Game $\text{DistGame}_1^{\Pi', \mathcal{A}}(\lambda)$ (Covert-IV Scheme) Init: 1. $k, dkey \xleftarrow{\\$} \text{aGen}(1^\lambda)$ 2. $ctr := 0$ 3. Give k to adversary Oracle $O(m, m_c)$: 1. $\langle \text{IV}, \text{C}, \text{T} \rangle := \text{aEnc}(k, dkey, ctr, m, m_c)$ 2. $ctr := ctr + 1$ 3. Return $\langle \text{IV}, \text{C}, \text{T} \rangle$ Output: Adversary outputs guess b'
--	--

Definition 2.5.1 (Distinguishing Advantage). For any compromised PPT adversary $\mathcal{A}$, we define the distinguishing advantage of $\mathcal{A}$ between the games DistGame_0 , DistGame_1 as:

$$\text{DistinguishAdv}_{\mathcal{A}}^{\Pi, \Pi'}(\lambda) = |\Pr[\text{DistGame}_0^{\Pi, \mathcal{A}}(\lambda) = 1] - \Pr[\text{DistGame}_1^{\Pi', \mathcal{A}}(\lambda) = 1]|.$$

We claim that Π' is secure if for all compromised PPT $\mathcal{A}$: $\text{DistinguishAdv}_{\mathcal{A}}^{\Pi, \Pi'}(\lambda) \leq \text{negl}(\lambda)$.

We remark that the distinguishing advantage intuitively measures how much better than random guessing $\mathcal{A}$ is at differentiating between the two experiments.

2.5.2 Preservation of AES-GCM's Security Guarantees

AES-GCM has been extensively analysed and is known to provide strong security guarantees, including confidentiality (IND-CPA) and integrity/authenticity (INT-CTXT). Although the security of the overt message m in the covert-IV scheme is broken trivially by a compromised adversary with k , we must ensure that the scheme preserves these guarantees against a standard adversary.

Definition 2.5.2 (Security Advantage). Let $\mathcal{P} \in \{\text{IND-CPA}, \text{IND-CTXT}\}$ denote the security properties of AES-GCM. Let $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi}(\lambda)$ denote the advantage

of a *standard* PPT adversary $\mathcal{A}$ in violating property $\mathcal{P}$ for protocol Π , and define $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi'}(\lambda)$ identically for the covert-IV scheme Π' .

We claim that Π' is secure if for all standard PPT adversaries $\mathcal{A}$: $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi}(\lambda) + \text{negl}(\lambda)$.

Intuitively, if any standard PPT adversary violates a security guarantee of the covert-IV scheme Π' , then there exists another standard PPT adversary that violates the same property of the standard AES-GCM protocol Π .

2.5.3 Confidentiality of the Covert Message

We require that an adversary who does not possess $dkey$ learns no information about the covert message $m_c \in \{0, 1\}^{|\text{IV}|}$ even under compromised conditions. The adversary provides two covert messages and outputs a bit b' indicating which covert message they believe has been encoded. The adversary is provided with the oracle:

$O(m, m_c^{(0)}, m_c^{(1)})$: encrypts plaintext m , embeds $m_c^{(b)}$, and returns $\langle \text{IV}, \text{C}, \text{T} \rangle$.

Game $\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda)$ (**Confidentiality of Covert Message in Covert-IV Scheme**)

Init:

1. $k, dkey \xleftarrow{\$} \text{aGen}(1^\lambda)$
2. $ctr := 0$
3. Give k to adversary
4. Challenge bit $b \xleftarrow{\$} \{0, 1\}$

Oracle $O(m, m_c^{(0)}, m_c^{(1)})$:

1. $\langle \text{IV}, \text{C}, \text{T} \rangle := \text{aEnc}(k, dkey, ctr, m, m_c^{(b)})$
2. $ctr := ctr + 1$
3. Return $\langle \text{IV}, \text{C}, \text{T} \rangle$

Output: Adversary outputs guess b'

Definition 2.5.3 (Confidentiality Advantage). For any compromised PPT adversary $\mathcal{A}$, we define the confidentiality advantage of $\mathcal{A}$ on game $\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda)$ as:

$$\text{ConfidentialAdv}_{\mathcal{A}}^{\Pi'}(\lambda) = \left| \Pr[\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right|.$$

We claim that Π' is secure if for all compromised PPT adversaries $\mathcal{A}$ without $dkey$: $\text{ConfidentialAdv}_{\mathcal{A}}^{\Pi'}(\lambda) \leq \text{negl}(\lambda)$.

2.6 Security Proofs

For clarity, we state some important details about standard primitives:

Definition 2.6.1 (Guarantees of Secure PRFs). A pseudorandom function family PRF is said to be secure if for every PPT adversary $\mathcal{B}$, the advantage $\text{PrfAdv}_{\mathcal{B}}(\lambda)$ in distinguishing $\text{PRF}_{dkey}(\cdot)$ from a uniformly random function with the same domain and range is negligible in the security parameter λ .

Definition 2.6.2 (XOR with Uniform Randomness). Let U be a uniformly random bitstring in $\{0, 1\}^n$, and let $M \in \{0, 1\}^n$ be any fixed bitstring. The distribution of $U \oplus M$ is uniform over $\{0, 1\}^n$. In particular, for any PPT adversary, the statistical distance between $U \oplus M$ and the uniform distribution is zero.

Next, we prove the three formal security requirements stated in the previous section.

Lemma 2.6.1 (Indistinguishability of Covert-IV and AES-GCM). *Assume $\text{PRF}_{dkey}(\cdot)$ is secure. Then for any compromised PPT adversary $\mathcal{A}$,*
 $\text{DistinguishAdv}_{\mathcal{A}}^{\Pi, \Pi'}(\lambda) \leq \text{negl}(\lambda)$.

Proof. Let $\mathcal{A}$ be any PPT adversary that can distinguish between DistGame_0 and DistGame_1 . We give a reduction from $\mathcal{A}$ to a PRF distinguisher $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine.

$\mathcal{B}$ participates in the standard PRF distinguishing game and is given oracle access to a function $O(\cdot)$, which is either $\text{PRF}_{dkey}(\cdot)$ or a truly random function with the same domain and range. $\mathcal{B}$ simulates the encryption oracle for $\mathcal{A}$ as follows:

- It samples $k \xleftarrow{\$} \text{Gen}(1^\lambda)$ and sends k to $\mathcal{A}$ and initialises a counter ctr .
- For each adaptive encryption query (m, m_c) issued by $\mathcal{A}$, $\mathcal{B}$ queries its oracle to obtain $y \leftarrow O(ctr)$, sets $\text{IV} := y \oplus m_c$, computes $(C, T) := \text{Enc}_k(\text{IV}, m)$, returns $\langle \text{IV}, C, T \rangle$ to $\mathcal{A}$, and increments ctr .
- $\mathcal{B}$ outputs the same guess b' as $\mathcal{A}$.

If $O(\cdot) = \text{PRF}_{dkey}(\cdot)$, then the view of $\mathcal{A}$ is distributed identically to its view in DistGame_1 , since the IVs are computed exactly as in the covert-IV construction. If $O(\cdot)$ is a truly random function, then for each counter value ctr , y is uniformly random and independent. By Definition 2.6.2, the distribution of $y \oplus m_c$ preserves this uniformity and hence the view of $\mathcal{A}$ is distributed identically to its view in DistGame_0 .

$\mathcal{B}$ outputs the same bit as $\mathcal{A}$; consequently, $\mathcal{B}$'s advantage in distinguishing $\text{PRF}_{dkey}(\cdot)$ from a truly random function is exactly equal to the advantage of $\mathcal{A}$ in distinguishing DistGame_0 from DistGame_1 . Formally, $\text{PrfAdv}_{\mathcal{B}}(\lambda) = \text{DistinguishAdv}_{\mathcal{A}}^{\Pi, \Pi'}(\lambda)$.

Since $\text{PRF}_{dkey}(\cdot)$ is secure, by Definition 2.6.1, $\text{PrfAdv}_{\mathcal{B}}(\lambda)$ is negligible. It follows that $\text{DistinguishAdv}_{\mathcal{A}}^{\Pi, \Pi'}(\lambda)$ must also be negligible. Therefore, the covert-IV construction is indistinguishable from AES-GCM under the stated assumptions. $\square$

We now show that the security guarantees of the covert-IV scheme reduce to those of AES-GCM with non-repeating random IVs against a standard adversary.

Lemma 2.6.2 (Preservation of AES-GCM Security Guarantees). *For all standard PPT adversaries $\mathcal{A}$, and $\mathcal{P} \in \{\text{IND-CPA}, \text{IND-CTXT}\}$, $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi}(\lambda) + \text{negl}(\lambda)$.*

Proof. Fix any standard PPT adversary $\mathcal{A}$ targeting a security property $\mathcal{P}$ of the covert-IV scheme Π' .

By Lemma 2.6.1, for any such adversary $\mathcal{A}$, the distribution of outputs $\langle \text{IV}, \text{C}, \text{T} \rangle$ produced using covertly generated IVs is computationally indistinguishable from the distribution obtained when IVs are sampled uniformly at random. Therefore, we define a hybrid game $\mathcal{H}$, where $\text{PRF}_{dkey}(\cdot)$ is replaced with a truly random function f . From the lemma we derive $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi'} \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \mathcal{H}} + \text{negl}(\lambda)$.

Observe that in $\mathcal{H}$, with uniform IV sampling, the experiment reduces to exactly the standard AES-GCM protocol Π , since XORing with m_c does not affect the random distribution (Definition 2.6.2). Therefore, it follows that $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \mathcal{H}} = \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi}$. By substitution, $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Pi}(\lambda) + \text{negl}(\lambda)$, as claimed. $\square$

Finally, we demonstrate that the covert-IV provides confidentiality for the covert message as well as indistinguishability.

Lemma 2.6.3 (Confidentiality of the Covert Message). *Assume $\text{PRF}_{dkey}(\cdot)$ is secure. Then for any compromised PPT adversary $\mathcal{A}$, without $dkey$, $\text{ConfidentialAdv}_{\mathcal{A}}^{\Pi'}(\lambda) \leq \text{negl}(\lambda)$.*

Proof. In $\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda)$, IVs are computed as $\text{IV}_t = \text{PRF}_{dkey}(\text{ctr}_t) \oplus m_c^{(b)}$. We define a hybrid game $\mathcal{H}$, where $\text{PRF}_{dkey}(\cdot)$ is replaced with a truly random function f , so $\text{IV}_t = f(\text{ctr}_t) \oplus m_c^{(b)}$.

By Lemma 2.6.1, the covert-IV construction is computationally indistinguishable from $\mathcal{H}$ under a secure PRF. Formally, for any PPT $\mathcal{A}$, $|\Pr[\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda) = 1] - \Pr[\mathcal{H}^{\mathcal{A}}(\lambda) = 1]| \leq \text{negl}(\lambda)$. This reduces the security of the covert-IV scheme to the case where the IVs are uniform and independent.

In $\mathcal{H}$, each IV is of the form $f(\text{ctr}_t) \oplus m_c^{(b)}$, where $f(\text{ctr}_t)$ is uniformly random. By Definition 2.6.2, XORing a uniformly random string with $m_c^{(b)}$ produces a uniformly random IV that is independent of the choice of b . Hence, $\mathcal{A}$ has no better strategy than random guessing, giving $\Pr[\mathcal{H}^{\mathcal{A}}(\lambda) = 1] = \frac{1}{2}$.

Combining these two steps via the hybrid argument, we have $\text{ConfidentialAdv}_{\mathcal{A}}^{\Pi'}(\lambda) = \Pr[\text{ConfGame}^{\Pi', \mathcal{A}}(\lambda) = 1] - \frac{1}{2} \leq \text{negl}(\lambda)$, as claimed. $\square$

Theorem 2.6.1 (Security of the Covert-IV Construction). *If $\text{PRF}_{dkey}(\cdot)$ is secure, then Π' achieves indistinguishability from Π , preserves the security properties of Π , and ensures confidentiality of covert messages.*

Proof. This follows from Lemma 2.6.1, Lemma 2.6.2, and Lemma 2.6.3. $\square$

2.7 IV Reuse

In Lemma 2.6.2, we relied on the security of Π to prove the security of Π' . However, standard AES-GCM is only secure if IVs do not repeat under the same key. We no longer assume unique IVs; instead, we quantify the probability of reuse.

We first recall and adopt the NIST's security requirement that the probability of IV reuse be $\leq 2^{-32}$. These guidelines also note that when IVs are chosen uniformly at random, the total number of AES-GCM encryptions under a single key should be limited to 2^{32} , at which point rekeying is recommended. We show that the scheme Π' is secure under these conditions.

Lemma 2.7.1 (Pairwise IV-Collision Bound). *Let $IV_1, \dots, IV_q$ be the IVs produced by the covert-IV construction Π' under a fixed symmetric key, for q encryptions. The probability that at least one IV is repeated satisfies*

$$\Pr [\exists i \neq j : IV_i = IV_j] \leq \binom{q}{2} 2^{-|IV|} + \text{negl}(\lambda).$$

Proof. An IV collision occurs if and only if there exists a pair of distinct indices $i \neq j$ such that $IV_i = IV_j$ for $1 \leq i < j \leq q$.

By construction, each IV is computed as $IV_t := \text{PRF}_{dkey}(ctr_t) \oplus m_{c,t}$. For any fixed pair $i \neq j$, $IV_i = IV_j$ implies $\text{PRF}_{dkey}(ctr_i) \oplus m_{c,i} = \text{PRF}_{dkey}(ctr_j) \oplus m_{c,j}$, which is equivalent to $\text{PRF}_{dkey}(ctr_i) \oplus \text{PRF}_{dkey}(ctr_j) = m_{c,i} \oplus m_{c,j}$.

We proceed by introducing a hybrid game $\mathcal{H}$ where PRF_{dkey} is replaced with a truly random function f . By Lemma 2.6.1, any PPT distinguisher can distinguish the covert-IV scheme from $\mathcal{H}$ with at most negligible probability $\text{negl}(\lambda)$.

Conditioned on $\mathcal{H}$, the outputs $f(ctr_i)$ and $f(ctr_j)$ are independent and uniformly random. Therefore, for any fixed pair (i, j) , the probability that $f(ctr_i) \oplus f(ctr_j) = m_{c,i} \oplus m_{c,j}$ holds is exactly $2^{-|IV|}$.

We apply the union bound over all unordered pairs (i, j) with $i \neq j$, which yields

$$\Pr [\exists i \neq j : IV_i = IV_j] \leq \binom{q}{2} 2^{-|IV|} + \text{negl}(\lambda).$$

This establishes a standard birthday-style collision bound, accounting for the PRF security of PRF_{dkey} . $\square$

The standard length of an IV in AES-GCM is 96 bits. Substituting $|IV| = 96$ into Lemma 2.7.1, we obtain $\Pr[\text{IV reuse after } q \text{ encryptions}] \leq \binom{q}{2} 2^{-96}$. For $q = 2^{32}$ encryptions under a fixed key, this yields $\binom{2^{32}}{2} 2^{-96} \leq 2^{63} \cdot 2^{-96} = 2^{-33}$, which satisfies the NIST requirement.

Consequently, we conclude that the covert-IV construction Π' preserves the guarantees of IV uniqueness required for AES-GCM security.

2.7.1 PRF Collisions

A related but distinct event is a collision of the PRF outputs: $\text{PRF}_{dkey}(ctr_i) = \text{PRF}_{dkey}(ctr_j)$. If this occurs, then $IV_i \oplus IV_j = (\text{PRF}_{dkey}(ctr_i) \oplus m_{c,i}) \oplus (\text{PRF}_{dkey}(ctr_j) \oplus m_{c,j}) = m_{c,i} \oplus m_{c,j}$. There are two cases to consider:

- If $\text{PRF}_{dkey}(ctr_i) = \text{PRF}_{dkey}(ctr_j)$ and $m_{c,i} = m_{c,j}$, then $IV_i = IV_j$. This corresponds to the IV reuse discussed in the previous section, which catastrophically breaks AES-GCM for those encryptions.
- If $\text{PRF}_{dkey}(ctr_i) = \text{PRF}_{dkey}(ctr_j)$ but $m_{c,i} \neq m_{c,j}$, then $IV_i \neq IV_j$. In this case, AES-GCM remains secure, but an observer who sees both IVs can learn $m_{c,i} \oplus m_{c,j}$, revealing information about the covert messages.

The probability of a PRF collision for any fixed pair (i, j) is at most $2^{-|IV|}$ by the same argument used in Lemma 2.7.1, and applying a union bound over all $\binom{q}{2}$ pairs produces an acceptably small probability of occurrence for practical parameters. Therefore, while PRF collisions can leak information about covert message XORs, they are sufficiently unlikely to be a concern.

2.7.2 Rekeying Policy

In accordance with NIST recommendations, the AES-GCM encryption key is refreshed after at most 2^{32} encryptions. We stipulate that, in our scheme, the double key $dkey$ and ctr are re-established simultaneously. This ensures that the same ctr is never used twice with the same $dkey$. Note that this rekeying no longer requires an out-of-band channel; it can now be coordinated through covert messages.

2.8 Robustness

In addition to indistinguishability and confidentiality, we require that the covert channel provide robustness. Informally, robustness captures both the integrity of covert messages and the soundness of their detection. That is, the receiver should accept only valid covert messages and reject modified or forged messages. Robustness does not guarantee delivery, as a compromised adversary may drop or reorder ciphertexts, but it ensures that any accepted covert message is authentic except with small probability.

In AES-GCM, integrity is provided by the authentication tag T computed over the ciphertext, IV, and associated data. However, under the compromised-receiver assumption, an adversary who learns the session key k can forge valid tags and arbitrarily modify (IV, C, T) without detection. Consequently, the AES-GCM tag does not protect the covert channel.

To address this, we extend Π' with an authentication mechanism for the covert payload. The double key $dkey$ is expanded via a key derivation function into independent subkeys $(dkey_{prf}, dkey_{mac}) := \text{KDF}(dkey)$. The key $dkey_{prf}$ is used for IV generation, while $dkey_{mac}$ is used to authenticate covert messages.

We encode covert messages as $m_c \in \{0, 1\}^{96}$ of the form $m_c := \text{data} \parallel \text{tag}$, where $\text{data} \in \{0, 1\}^{64}$ is the covert payload and $\text{tag} \in \{0, 1\}^{32}$ is defined as $\text{tag} := \text{MAC}_{dkey_{mac}}(\text{data})$.

To transmit a covert message, Alice constructs m_c as above. If no covert message is intended, she instead samples $m_c \xleftarrow{\$} \{0, 1\}^{96}$. In both cases, the IV is computed as $\text{IV} := \text{PRF}_{dkey_{prf}}(ctr) \oplus m_c$, and encryption proceeds as in Π' .

Upon receiving $\langle \text{IV}, C, T \rangle$, Bob computes $m_c^* := \text{PRF}_{dkey_{prf}}(ctr) \oplus \text{IV}$ and parses it as $(\text{data}, \text{tag})$. He recomputes $\text{tag}' := \text{MAC}_{dkey_{mac}}(\text{data})$ and accepts the covert message if and only if $\text{tag}' = \text{tag}$. Otherwise, he rejects.

Definition 2.8.1 (Robustness of Π'). Let $\mathcal{A}$ be any compromised PPT adversary interacting with the scheme. Define the robustness advantage as $\text{RobustAdv}_{\mathcal{A}}^{\Pi'} := \Pr[\text{AcceptInvalid}]$, where AcceptInvalid is the event that the receiver accepts a covert message $(\text{data}, \text{tag})$ that was not honestly generated using $dkey_{mac}$. We say that Π' is robust if for all PPT adversaries $\mathcal{A}$, $\text{RobustAdv}_{\mathcal{A}}^{\Pi'} \leq \epsilon$, for small ϵ determined by the tag length.

In the context of the covert channel, we do not claim full asymptotic security because the channel is small and the tag length is fixed. Rather, the bound ϵ quantifies a concrete probability of failure per attempt, which remains acceptably small for this use case, as the adversary does not have oracle access to $dkey_{mac}$.

Lemma 2.8.1 (Robustness of Covert-IV Scheme). *Let Π' be the covert-IV scheme with MAC-authenticated covert messages $m_c = \text{data} \parallel \text{tag}$, where $\text{tag} := \text{MAC}_{dkey_{mac}}(\text{data})$ and $dkey_{mac}$ is unknown to the adversary. Then for any compromised PPT adversary $\mathcal{A}$ making at most q attempts, $\text{RobustAdv}_{\mathcal{A}}^{\Pi'} \leq q \cdot 2^{-|\text{tag}|}$, where $|\text{tag}|$ is the tag length.*

Proof. Consider any ciphertext processed by the receiver. The reconstructed value $m_c = \text{PRF}_{dkey_{prf}}(ctr) \oplus \text{IV}$ yields a candidate pair $(\text{data}, \text{tag})$. For any pair not honestly generated using $dkey_{mac}$, the value of tag is independent of $\text{MAC}_{dkey_{mac}}(\text{data})$. EUF-CMA guarantees that any forgery or random attempt succeeds only with probability at most $2^{-|\text{tag}|}$ per query (Bellare et al., 1996; Katz and Lindell, 2014). Thus, over q attempts, a union bound gives $\Pr[\text{AcceptInvalid}] \leq q \cdot 2^{-|\text{tag}|}$.

This shows that in our robust scheme the probability of accepting any invalid covert message is bounded by 2^{-32} per attempt. $\square$

Note that this bound captures both adversarial forgeries/modifications, and accidental acceptance of randomly generated IVs. The former is a computational bound that relies on the unforgeability of the MAC, while the latter arises from random collisions and is information-theoretic. In both cases, the success probability is bounded by $2^{-|\text{tag}|}$ per attempt. For practical parameters, both probabilities remain small, ensuring reliable integrity and detection of covert messages. A longer tag would increase the robustness of the covert channel at the expense of reducing the amount of covert data embedded in each message.

2.9 Attacks

The security guarantees of the AES-GCM protocol have been studied extensively under standard conditions. Consequently, we focus our analysis on attacks directed at the covert message or only possible under compromised assumptions.

2.9.1 Replay

A replay attack occurs when an adversary retransmits a previously observed $\langle IV, C, T \rangle$ to induce acceptance by the receiver. In Π' , such attacks are mitigated by the use of a monotonically increasing counter ctr , which is incremented for each message and never reused under the same $dkey$. Replaying an earlier message produces a mismatch between the expected PRF output and the received IV, causing the computed tag' to fail verification. Consequently, replayed messages are rejected except with a small probability corresponding to a PRF collision or accidental acceptance, which is bounded by 2^{-32} per attempt, consistent with the robustness analysis in Section 2.8.

2.9.2 Re-encryption

Under the compromised-sender assumption, the adversary is prevented from dictating the randomness used by Alice during encryption. The logical next step for an adversary would be to circumvent this restriction by replacing the original ciphertext with a re-encryption under an adversarially selected randomness. This attack is trivial for Eve to mount under the compromised-receiver assumption.

Specifically, given an intercepted $\langle IV, C, T \rangle$ sent from Alice to Bob, the adversary proceeds as follows:

- Eve compels the receiver Bob to reveal k , and decrypts $m := \text{Dec}_k(\langle IV, C, T \rangle)$.
- Eve samples a new initialisation vector $IV' \xleftarrow{\$} \{0, 1\}^{|IV|}$ such that $IV' \neq IV$, re-encrypts $(C, T) := \text{Enc}_k(IV', m)$, and sends only $\langle IV', C, T \rangle$ to Bob.
- Bob successfully decrypts m , but verification of the covert authentication tag in m_c fails.

Under this attack, provided that IV' is a unique initialisation vector under k , the security properties of AES-GCM are not violated as this is equivalent to two independent executions of the protocol.

The covert-IV scheme has a mechanism to identify this interference as the tag verification will fail, indicating the presence of tampering. While re-encryption attacks cannot be prevented, their effects are detectable in Π' , thereby preserving the robustness of the covert channel.

2.9.3 Message Dropping

A subtle attack involves selective message dropping. The covert-IV scheme Π' requires a monotonically increasing shared counter ctr that is incremented locally after sending

or receiving a message. We consider the following attack:

- Alice produces $\langle IV, C, T \rangle$ encoding m_c under ctr_i .
- The Adversary intercepts this communication and withholds it instead of delivering it to Bob.
- When Bob attempts to send a covert message, his counter has not advanced, causing him to produce a second covert message under the same counter value ctr_i .
- The adversary can exploit this collision as discussed in Section 2.7.1.

This attack stems from the use of a single covert key and a single counter in both communication directions. To mitigate it, we refine the construction by introducing direction-specific covert keys.

Let $dkey_A$ denote the covert key used by Alice when sending to Bob, and $dkey_B$ denote the covert key used by Bob when sending to Alice. Both parties share these keys and can therefore decrypt the covert messages they receive. We retain a single shared, monotonically increasing counter ctr , but derive IVs directionally as follows:

$$IV = \begin{cases} \text{PRF}_{dkey_A}(ctr) \oplus m_c & \text{for messages from Alice to Bob,} \\ \text{PRF}_{dkey_B}(ctr) \oplus m_c & \text{for messages from Bob to Alice.} \end{cases}$$

Under this modification, even if the adversary successfully suppresses a message and causes both parties to use the same counter value ctr_i , the resulting PRF outputs remain independent provided $dkey_A \neq dkey_B$. Therefore, $\text{PRF}_{dkey_A}(ctr_i) \neq \text{PRF}_{dkey_B}(ctr_i)$ except with probability 2^{-33} of a PRF collision.

Desynchronisation

Another weakness that the message dropping attack exploits is the synchronisation of the shared counter. If Alice increments her counter after sending a message that Bob never receives, subsequent ciphertexts will be generated using counter values that differ from Bob's expectation.

To mitigate this attack, the receiver should attempt covert verification using a series of counter values near the current counter state. That is, upon receiving a ciphertext, Bob checks whether the tag validates under any counter value within a small range around his current ctr . If validation succeeds, Bob can resynchronize his counter accordingly. This windowed covert verification strategy preserves security while allowing resynchronisation.

2.9.4 Timing attacks

Timing attacks exploit variations in execution time to infer secret information from cryptographic implementations (Kocher, 1996). The experimental results of Figure 2.2 show the additional computational cost introduced by encoding a 96-bit covert message across varying overt message sizes. While the relative overhead decreases as the overt

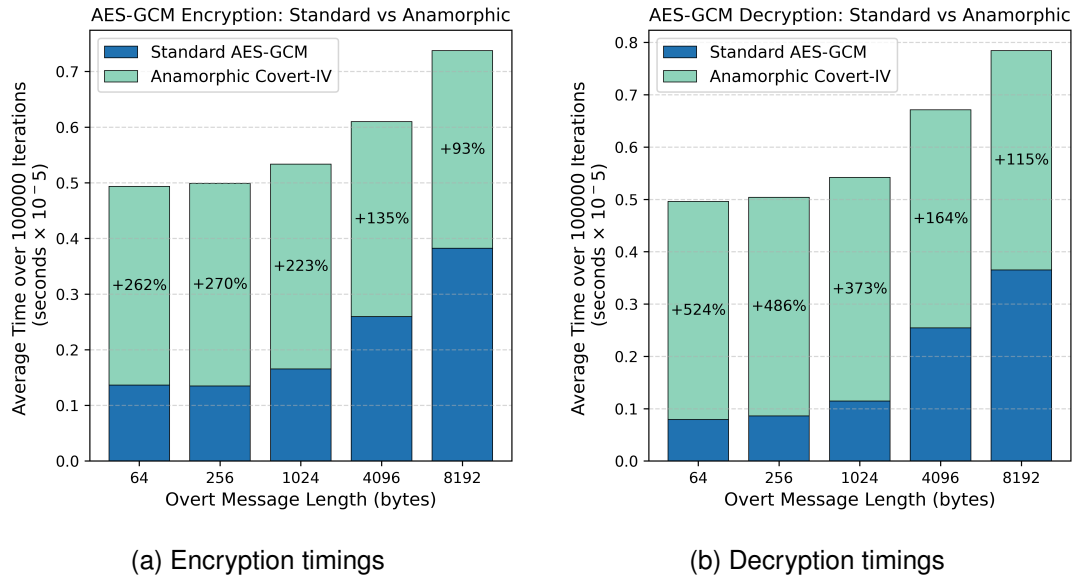


Figure 2.2: Timing overhead introduced by the covert-IV scheme compared to standard AES-GCM across varying overt message sizes. The percentage labels shown within each bar represent the relative overhead introduced by the anamorphic implementation compared to the standard implementation. All experiments in this paper were conducted on the Eddie HPC cluster using GPU-enabled nodes (Intel Xeon Gold 6338 CPU, 64 cores, 768 GB RAM, NVIDIA A100 GPUs) (Edinburgh Compute and Data Facility, 2021). Computations used Python’s *cryptography* AES-GCM implementation using 256-bit keys, and were CPU-only to reflect realistic conditions. The PRF was modelled using HMAC-SHA256. Timing was measured using Python’s *time.perf_counter()* and averaged over 100,000 iterations.

message length increases, the absolute cost remains approximately constant. Across the evaluated message sizes, the average additional cost of anamorphic encryption was $3.6\mu\text{s}$, while the average additional cost of anamorphic decryption was $4.2\mu\text{s}$.

Given that typical network jitter and operating system scheduling noise are several orders of magnitude larger than this overhead, it is unlikely in practice that a remote adversary could reliably distinguish anamorphic messages from standard AES-GCM messages using timing analysis alone. Previous work has shown that remote timing attacks must overcome significant noise introduced by network transmission latency and system scheduling (Brumley and Boneh, 2003; Van Goethem and Joosen, 2020). Consequently, timing-based detection does not appear to pose a practical threat to the covert channel under realistic deployment conditions.

2.10 Discussion

The proposed covert-IV scheme improves upon the limitations of earlier anamorphic constructions. In particular, it overcomes the problem of low bandwidth in rejection sampling-based schemes: embedding 64 bits of covert data in such constructions would require an expected 2^{64} trials, whereas the covert-IV scheme achieves this bandwidth

directly per message. Additionally, unlike the Naor-Yung framework, which requires the double key to be generated simultaneously with the public and secret key pair, our scheme allows parties to initiate covert communication at any time after establishing a double key. This flexibility enables deployment on existing ciphertext systems without prior modifications. The covert channel preserves stealth, as all hidden information is embedded within protocol-mandated fields, avoiding increases in message size or any observable deviation from normal behaviour.

A further advantage of the construction is that it can be implemented to add an additional layer of obscurity to the identities of the communicating parties. Consider three participants: Alice, Bob, and Charlie. Suppose Alice and Charlie wish to exchange covert messages but communicate only through their ordinary encrypted interactions with Bob. If we assume that Alice and Charlie can observe each other's communications with Bob, they can embed covert messages within the IVs of their individual communications with Bob. More precisely, since the IV is transmitted unencrypted, extraction and verification of the covert message requires only knowledge of the shared double key $dkey$, and not the symmetric key k . Counters can be synchronised from the sequence of messages each party sends to Bob, allowing Alice and Charlie to maintain a covert channel without direct communication.

In this scenario, Bob does not even need to be aware that any covert communication is taking place, which provides an additional degree of deniability for all parties involved. Moreover, because the covert messages are embedded in traffic directed to Bob rather than exchanged directly between Alice and Charlie, it becomes more difficult for an external observer to establish a direct link between the two conspiring parties.

Despite these advantages, the applicability of the covert-IV construction depends on how AES-GCM is implemented in practice. AES-GCM only requires that IVs be non-repeating under a certain symmetric key k , it does not require them to be random. Consequently, many real-world protocols derive IVs according to a fixed, deterministic pattern. Protocols that employ such IV sequences do not provide a usable channel for covert communication. In these cases, IV values are fully predictable to the adversary and need not be transmitted, as both parties can independently compute the next IV in the sequence.

Consequently, the covert-IV construction is best suited to protocols that explicitly transmit IVs and allow random generation. Within this subset of protocols, the scheme offers a practical, low-bandwidth covert channel that preserves standard security guarantees. More generally, the approach is not specific to AES-GCM: the same technique can be applied to any symmetric encryption scheme that transmits a randomly sampled IV or nonce as part of the ciphertext.

Chapter 3

Anamorphic Signal

Although the covert-IV scheme provides a mechanism for transmitting a significant number of covert bits per message, it also faces barriers to implementation in the real world, as we have discussed. Now, we suggest a construction with a smaller covert channel; this channel is built into the foundations of the protocol and is harder to prevent.

3.1 Preliminaries

Asymmetric Key Generation

Let $\mathbb{G}$ be a cyclic group of prime order q , where q is a large prime defining the order of the group, $\mathbb{Z}_q$ is the set of integers modulo q , and $g \in \mathbb{G}$ is a generator of $\mathbb{G}$.

Many secure messaging protocols derive shared secrets using the Diffie-Hellman key agreement over $\mathbb{G}$. A user samples a private key $sk \xleftarrow{\$} \mathbb{Z}_q$ and computes the corresponding public key $pk = g^{sk}$. Key generation is canonical: each private key deterministically maps to exactly one valid public key. To establish a shared secret, two users exchange ephemeral public keys $pk_A = g^{sk_A}$ and $pk_B = g^{sk_B}$ and compute $K_{AB} = g^{sk_A sk_B} = pk_A^{sk_B} = pk_B^{sk_A}$. Thus, both parties derive the same shared value K_{AB} .

Rejection Sampling

Rejection sampling is a method for selecting samples from a space by repeatedly drawing candidates until one satisfies a rejection condition. If each trial succeeds with probability p , the number of trials follows a geometric distribution with expected value $1/p$ (Wikipedia Contributors, 2004).

Formally, let $\mathcal{C}$ denote the space of ciphertexts generated by an encryption algorithm $\text{Enc}_k(m; r)$ for fixed message m and randomness r . Let $P(c) \in \{0, 1\}$ be a predicate that checks whether a candidate $c \in \mathcal{C}$ satisfies a desired property. The rejection sampling process is as follows:

1. Draw fresh randomness r and compute $c := \text{Enc}_k(m; r)$.

2. If $P(c) = 1$, output c ; else, repeat from step 1.

Horel et al. (Horel et al., 2019) observed that this process can embed information into ciphertexts by defining the predicate so that only samples encoding a desired bit pattern are accepted. If the predicate enforces that ℓ bits match a chosen string, the success probability is $p = 2^{-\ell}$ and the expected number of trials is 2^ℓ .

3.2 Signal Protocol

The Signal protocol is a widely deployed secure messaging framework that provides post-compromise and forward secrecy for asynchronous communication. Forward secrecy ensures that the compromise of long-term keys does not reveal past communications, while post-compromise security guarantees that security can be restored in future communications after a compromise. It is used in messaging systems such as Signal, WhatsApp, and Facebook Messenger and is designed to operate in environments where parties may be online intermittently. The protocol is composed of three main components: the Extended Triple Diffie–Hellman (X3DH) handshake protocol, which establishes an initial shared secret between communicating parties; the Double Ratchet algorithm, which provides ongoing key evolution and forward secrecy for message encryption; and an authenticated encryption scheme, typically AES-GCM, which is used to encrypt individual messages (Cohn-Gordon et al., 2019). To minimise the damage of key compromise, Signal uses separate symmetric encryption keys for each direction of communication. For example, if Alice encrypts a message with her sending key, Bob can decrypt it with his receiving key.

For brevity, the descriptions below intentionally abstract from several low-level implementation details.

3.2.1 X3DH Handshake

The X3DH (Extended Triple Diffie–Hellman) protocol is an authenticated key agreement mechanism that allows two parties, Alice and Bob, to establish a shared secret even if Bob is offline. Bob publishes a set of public keys to a server: a long-term identity key, a signed medium-term prekey, and a set of one-time prekeys. When Alice wants to initiate a conversation, she retrieves these keys from the server.

Alice generates an ephemeral public key pair and combines it with her own identity key and Bob’s keys through three (or optionally four) Diffie–Hellman operations. The resulting shared secrets are concatenated and processed through a key derivation function KDF_{root} to produce the initial root key and the first sending chain key. We denote these keys as rk_x and $ck_{x,y}^{\text{send}}$, where x is the number of asymmetric ratchets that have taken place, and y is the number of symmetric ratchets applied to the send chain since the last asymmetric ratchet. We will describe these ratchets in more detail later.

Next, Alice sends an initial message containing her public keys, the identifiers of Bob’s keys used, and an encrypted payload. Upon receiving Alice’s message, Bob uses his corresponding private keys to recompute the same Diffie–Hellman exchange and discovers rk_x . This equivalence follows from the Diffie–Hellman property: for any key

pairs (sk_A, pk_A) and (sk_B, pk_B) , $DH(sk_A, pk_B) = DH(sk_B, pk_A)$. Bob later derives his receiving chaining key $ck_{0,0}^{recv}$ from Alice's first message through the asymmetric ratchet.

3.2.2 Symmetric Ratchet

Following the X3DH handshake, Alice and Bob possess a shared root key rk_x and a direction-specific chaining key. The symmetric ratchet generates a fresh encryption key for each message sent or received. Each communication direction maintains an independent symmetric chain.

Let $ck_{x,y}^d$ denote the chaining key for direction $d \in \{\text{send}, \text{recv}\}$ after x asymmetric rattles and y symmetric ratchet steps. To send or receive a message, the current chaining key is input to a key derivation function KDF_{mess} that deterministically outputs a message key and the next chaining key: $(mk_{x,y}^d, ck_{x,y+1}^d) \leftarrow KDF_{\text{mess}}(ck_{x,y}^d)$. The message key $mk_{x,y}^d$ is used exactly once to encrypt or decrypt a single message, after which it is erased. The updated chaining key $ck_{x,y+1}^d$ is retained for the next ratchet.

The symmetric ratchet is able to provide forward secrecy within each asymmetric ratchet period due to the properties of KDF constructions. However, if an attacker learns the current chain key, they can compute all future keys. Therefore, we need asymmetric rattling to provide post-compromise security.

3.2.3 Asymmetric Ratchet

The asymmetric ratchet periodically injects fresh entropy into the shared state and produces new root keys. Unlike the symmetric ratchet, which advances on every message, the asymmetric ratchet is triggered when a party receives a message containing a previously unseen ratchet public key.

Let $(sk_{A,x}^{\text{RCH}}, pk_{A,x}^{\text{RCH}})$ denote Alice's ratchet key pair associated with asymmetric ratchet index x , and similarly $(sk_{B,x}^{\text{RCH}}, pk_{B,x}^{\text{RCH}})$ for Bob. Each message header contains the sender's current ratchet public key. Upon receiving a message, the recipient compares the included public key with the last stored key for that peer. If the key differs, an asymmetric ratchet step is performed.

Suppose Alice receives a message from Bob containing a new ratchet public key $pk_{B,x+1}^{\text{RCH}}$. Alice then performs the following steps.

1. **First Diffie-Hellman Exchange.** Alice computes $DH_1 = DH(sk_{A,x}^{\text{RCH}}, pk_{B,x+1}^{\text{RCH}})$. This shared secret is combined with the previous root key to derive a new root key and the first receiving chain key: $rk', ck_{x+1,0}^{recv} \leftarrow KDF_{\text{root}}(rk_x \parallel DH_1)$. A symmetric ratchet of the receiving chain provides the message key $mk_{x+1,0}^{recv}$ for subsequent decryption.
2. **Ratchet Key Generation.** Alice generates a new ratchet key pair $(sk_{A,x+1}^{\text{RCH}}, pk_{A,x+1}^{\text{RCH}})$.
3. **Second Diffie-Hellman Exchange.** Alice computes $DH_2 = DH(sk_{A,x+1}^{\text{RCH}}, pk_{B,x+1}^{\text{RCH}})$ and derives the sending chain key: $rk_{x+1}, ck_{x+1,0}^{\text{send}} \leftarrow KDF_{\text{root}}(rk' \parallel DH_2)$.

A symmetric ratchet of the sending chain provides the message key $mk_{x+1,0}^{\text{send}}$ for subsequent encryption.

The first Diffie–Hellman computation derives the receiving chain for messages encrypted under the peer’s new ratchet key. In contrast, the second derives the sending chain associated with the newly generated local ratchet key. Bob performs the same computations symmetrically when processing Alice’s subsequent message containing $pk_{A,x+1}^{\text{RCH}}$.

A standard adversary who compromises past root or chaining keys but does not possess the current ratchet private key cannot compute the Diffie–Hellman secrets DH_1 or DH_2 . Therefore, they cannot derive the updated root key rk_{x+1} or any subsequent message keys. Consequently, the asymmetric ratchet provides post-compromise security by ensuring that confidentiality is restored after the next honest asymmetric ratchet exchange.

Note that the asymmetric ratchet is advanced whenever the direction of communication changes. In particular, upon receiving a message containing a new public key from Alice, Bob also generates a new ratchet public key before sending his next message.

3.3 Covert-Keyspace Scheme

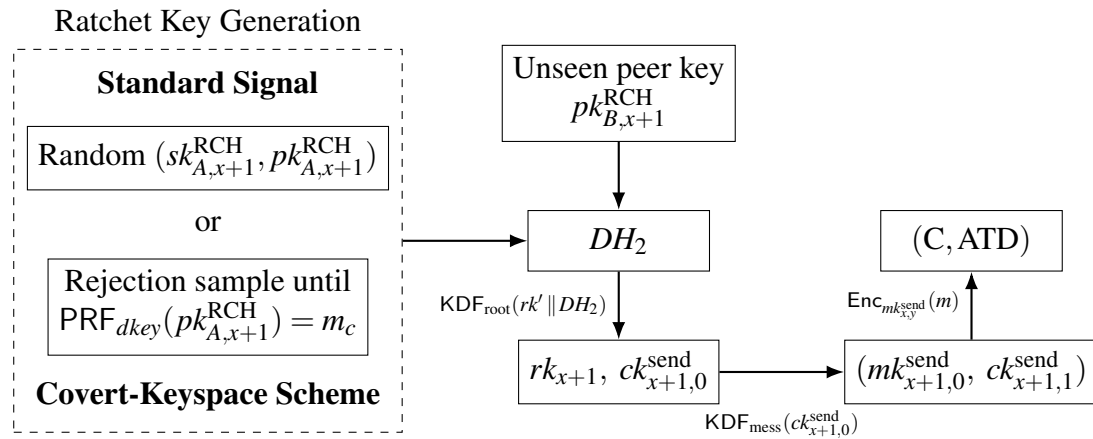


Figure 3.1: Visualisation of part of the asymmetric ratchet in standard Signal and the covert-keyspace scheme for Alice. Note that rk' is computed in the first Diffie–Hellman exchange (not shown here), and (C, ATD) represents the computed ciphertext of overt message m and additional transmitted data such as the IV and authentication tag. Alice transmits $\langle pk_{A,x+1}^{\text{RCH}}, C, ATD \rangle$.

In this section, we present our second construction, which extends the Signal protocol to enable the transmission of covert messages without breaking the security guarantees of the protocol. We will refer to this scheme as the covert-keyspace scheme.

As in our previous construction, Alice and Bob pre-share a secret key $dkey$ through an out-of-band channel. Next, they perform the X3DH handshake as standard to create the initial channel. In typical bidirectional communication where messages alternate

between participants, the asymmetric ratchet advances once per direction change; Alice can use these asymmetric ratchets to embed her covert message. Instead of selecting her ratchet key pair uniformly at random, Alice rejection samples candidate key pairs until she obtains a public key $pk_{A,x+1}^{\text{RCH}}$ satisfying $\text{PRF}_{dkey}(pk_{A,x+1}^{\text{RCH}}) = m_c$, where m_c denotes the covert message to be transmitted.

Alice then completes the asymmetric ratchet as specified by the protocol and shown in Figure 3.1, which includes transmitting pk_A^{RCH} to Bob in the message header. Bob performs the asymmetric key updates, derives the corresponding root and chain keys in the standard way, and decrypts the overt message. Additionally, using $dkey$, Bob evaluates $\text{PRF}_{dkey}(pk_{A,x+1}^{\text{RCH}})$ to recover the covert message m_c .

The asymmetric ratchet creates a channel for ongoing covert communication, as it is triggered whenever the direction of communication changes. Not every message is capable of carrying a distinct covert message. In particular, if Alice transmits multiple consecutive messages prior to receiving a response from Bob, all messages within this sequence encode the same covert information, since they are sent in the same communication direction and retransmit an identical public key.

Note that a similar scheme exists to insert a covert message into the X3DH handshake as this protocol also requires the initiator to generate an ephemeral key pair. We do not consider the scheme as it only provides a one-time covert channel.

3.4 Syntax

We model an abstracted Signal messaging protocol and an anamorphic extension to it. Our abstraction hides low-level details but exposes the essential public values and state transitions needed for security arguments.

Let $\Sigma = (\text{Handshake}, \text{Send}, \text{Recv})$ denote an abstracted messaging protocol capturing the X3DH handshake and Double Ratchet behaviour of Signal, using AES-GCM for symmetric encryption.

For a security parameter λ :

- $\text{Handshake}(1^\lambda)$: Executes the X3DH handshake between Alice and Bob and returns initial local states (st_A, st_B) . Local state st_A contains: $rk_0, ck_{0,0}^{\text{send}}, ck_{0,0}^{\text{recv}}, sk_{A,0}^{\text{RCH}}, pk_{B,0}^{\text{RCH}}$, where rk_0 is the initial root key, $ck_{0,0}^{\text{send}}$ and $ck_{0,0}^{\text{recv}}$ are the initial chaining keys, $sk_{A,0}^{\text{RCH}}$ is Alice's initial ratchet private key, and $pk_{B,0}^{\text{RCH}}$ is the last observed peer ratchet key from Bob. Similarly, st_B contains the equivalent values for Bob.
- $\text{Send}(st, m)$: If an asymmetric ratchet has been triggered (a new peer key was received in the previous message), follow steps 2-3 of the asymmetric ratchet as described in Section 3.2.3 and add the new ratchet public key pk_{x+1}^{RCH} to the message header. Else perform a symmetric ratchet on the sending chain as described in Section 3.2.2 and include the current, unchanged ratchet public key pk_x^{RCH} in the message header.

Then encrypt $(C, \text{ATD}) := \text{Enc}_{mk^{\text{send}}}(m)$, where ATD denotes additional transmitted data. The sender transmits a message tuple $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle$ and updates its local state to st' .

- $\text{Recv}(st, \langle pk^{\text{RCH}}, C, \text{ATD} \rangle)$: If pk^{RCH} differs from the stored peer key, perform the first step of the asymmetric ratchet as described in Section 3.2.3. Else, perform a symmetric ratchet on the receiving chain.

Then decrypt $m := \text{Dec}_{mk^{\text{recv}}}(C, \text{ATD})$ and updates its local state to st' .

Note that the inner-workings of the asymmetric ratchet occur across both Send and Recv.

Definition 3.4.1 (Covert-Keyspace Scheme). Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ be a keyed secure pseudorandom function, where λ is the security parameter, $n = 256$ (standard length of ephemeral keys in Signal) (Perrin et al., 2025) and ℓ is the length of the covert message m_c . Let $dkey$ be shared out-of-band. The Scheme Σ' extends Σ as follows:

- $\text{aHandshake}(1^\lambda)$: As Handshake but additionally returns a double key $dkey$.
- $\text{aSend}(st, dkey, m, m_c)$: *Precondition: a new asymmetric ratchet is being performed.* To embed a covert message m_c :

1. Sample ratchet private key $sk_{x+1}^{\text{RCH}} \xleftarrow{\$} \mathbb{Z}_q$ and compute $pk_{x+1}^{\text{RCH}} := g^{sk_{x+1}^{\text{RCH}}}$
2. If $\text{PRF}_{dkey}(pk_{x+1}^{\text{RCH}}) = m_c$, select pk_{x+1}^{RCH} ; else, repeat from step 1.

Then proceed as in Send to compute (C, ATD) . The sender transmits a message tuple $\langle pk_{x+1}^{\text{RCH}}, C, \text{ATD} \rangle$ and updates its local state to st' .

- $\text{aRecv}(st, dkey, \langle pk^{\text{RCH}}, C, \text{ATD} \rangle)$: Proceed as in Recv. If pk^{RCH} differs from the stored peer key, then $m_c := \text{PRF}_{dkey}(pk^{\text{RCH}})$ and update local state to st' .

Just as we noted for the covert-IV scheme, the covert-keyspace can also be operated with correctness in intermediate modes: $\Sigma'_{\text{aSend,Recv}}$ and $\Sigma'_{\text{Send,aRecv}}$. For the remainder of this chapter, we focus on the security and robustness of the full anamorphic scheme Σ' as the guarantees of the intermediate modes reduce to the guarantees of Σ and Σ' .

3.5 Security Definition

The security goals of the covert-keyspace scheme closely parallel those of the covert-IV construction defined in Section 2.5. We therefore reuse the same game-based security framework, adapting it to the Signal protocol and to public-key selection in place of IV generation.

To simplify the following security claims, we make the following assumptions.

- **Compromised Advantage.** In the following games, we satisfy the compromised-sender assumptions by allowing the adversary to choose the overt

message to be sent, and making the randomness used in each ratchet and encryption, pk^{RCH} and ATD , available in the returned tuple. Similarly, we do not limit the adversary’s ability to choose the recipient of the message.

To model the compromised-receiver assumption, we provide the adversary with the internal states (st_A, st_B) , which include the messaging, chaining, and root keys. Using the secret keys that these states provide, the adversary can recover the new states after a symmetric or asymmetric ratchet update.

- **Non-Repeating Ratchet Keys.** We assume that the ratchet public keys generated by the covert-keyspace scheme do not repeat across distinct ratchet epochs; for each index x , the pk_x^{RCH} must be distinct. This assumption is required for Signal’s standard security guarantees to hold. The probability of key reuse is considered in Section 3.7.

3.5.1 Indistinguishability

We define an oracle and distinguishing games analogous to Section 2.5.1 in which the adversary attempts to distinguish executions of the standard Signal protocol from executions of the covert-keyspace scheme.

As shown in Definition 3.4.1, all elements of the covert-keyspace execution are identical except for the asymmetric ratchet steps. Consequently, to establish indistinguishability between Σ and Σ' , it suffices to show that the asymmetric ratchet in Σ' is computationally indistinguishable from the same ratchet in Σ against a compromised adversary.

Game $\text{DistGame}_0^{\Sigma, \mathcal{A}}(\lambda)$ (Standard Signal Asymmetric Ratchet)	Game $\text{DistGame}_1^{\Sigma', \mathcal{A}}(\lambda)$ (Covert-KeySpace Scheme Asymmetric Ratchet)
Init: 1. $(st_A, st_B) \xleftarrow{\\$} \text{Handshake}(1^\lambda)$ 2. Give (st_A, st_B) to adversary	Init: 1. $(st_A, st_B, dkey) \xleftarrow{\\$} \text{aHandshake}(1^\lambda)$ 2. Give (st_A, st_B) to adversary $\mathcal{A}$
Oracle $O(m, m_c \in \{0, 1\}^\ell)$: 1. $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle, st'_A := \text{send}(st_A, m)$ 2. $st_A := st'_A$ 3. Return $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle$	Oracle $O(m), m_c \in \{0, 1\}^\ell$: 1. $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle, st'_A := \text{aSend}(st_A, dkey, m, m_c)$ 2. $st_A := st'_A$ 3. Return $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle$
Output: Adversary outputs guess b'	Output: Adversary outputs guess b'

Formally, by adapting Definition 2.5.1, the covert-keyspace scheme is indistinguishable from standard Signal if for all compromised PPT adversaries $\mathcal{A}$: $\text{DistinguishAdv}_{\mathcal{A}}^{\Sigma, \Sigma'}(\lambda) \leq \text{negl}(\lambda)$, where $\text{DistinguishAdv}_{\mathcal{A}}^{\Sigma, \Sigma'}(\lambda) = |\Pr[\text{DistGame}_0^{\Sigma, \mathcal{A}}(\lambda) = 1] - \Pr[\text{DistGame}_1^{\Sigma', \mathcal{A}}(\lambda) = 1]|$.

3.5.2 Preservation of Signal's Security Guarantees

The Signal protocol has been extensively analysed and is widely understood to achieve strong security guarantees consistent with authenticated and confidential channel establishment (ACCE)-style security. In particular, these include: message confidentiality, message authenticity and integrity, forward secrecy, and post-compromise security. We must ensure that the covert-keyspace scheme preserves these guarantees against a standard adversary.

Definition 3.5.1 (Security Advantage). Let $\mathcal{P} \in \{\text{CONF}, \text{INT}, \text{FS}, \text{PCS}\}$ denote message confidentiality, integrity/authenticity, forward secrecy, and post-compromise security, respectively. Let $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma}(\lambda)$ denote the advantage of a standard PPT adversary $\mathcal{A}$ in violating property $\mathcal{P}$ in Σ , and define $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma'}(\lambda)$ analogously.

We claim that the covert-keyspace scheme Σ' preserves security property $\mathcal{P}$ of Signal if for all standard PPT adversaries $\mathcal{A}$: $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma}(\lambda) + \text{negl}(\lambda)$.

3.5.3 Confidentiality of the Covert Message

We require confidentiality of the covert message m_c against compromised adversaries that do not possess the shared double key $dkey$. This property is defined via an oracle and a left-right game analogous to $\text{ConfGame}^{\Pi', \mathcal{A}}$ in which the adversary selects covert messages of length ℓ and attempts to distinguish which one is encoded into the public keys appearing in the protocol transcript.

Game $\text{ConfGame}^{\Sigma', \mathcal{A}}(\lambda)$ (**Covert-Message Confidentiality**)

Init:

1. $(st_A, st_B, dkey) \xleftarrow{\$} \text{aHandshake}(1^\lambda)$
2. Give (st_A, st_B) to adversary $\mathcal{A}$
3. Challenge bit $b \xleftarrow{\$} \{0, 1\}$

Oracle $O(m, m_c^{(0)}, m_c^{(1)})$:

1. $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle, st'_A := \text{aSend}(st_A, dkey, m, m_c^{(b)})$
2. $st_A := st'_A$
3. Return $\langle pk^{\text{RCH}}, C, \text{ATD} \rangle$

Output: Adversary outputs guess b'

By adapting Definition 2.5.3, we formally say that the covert-keyspace scheme protects the confidentiality of the covert message m_c if for all compromised PPT adversaries $\mathcal{A}$: $\text{ConfidentialAdv}_{\mathcal{A}}^{\Sigma'}(\lambda) \leq \text{negl}(\lambda)$, where $\text{ConfidentialAdv}_{\mathcal{A}}^{\Sigma'}(\lambda) = \left| \Pr[\text{ConfGame}^{\Sigma', \mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right|$.

3.6 Security Proofs

Here, we will consider the theoretical proofs of this scheme. As alluded to in Section 3.1, rejection sampling requires significant computational resources; we will consider practical limits in Section 3.8.

We proceed by proving the indistinguishability of the rejection sampled keys from the properties of public keys.

Definition 3.6.1 (Uniform Distribution of Public Keys). Let $\mathbb{G}$ be a cyclic group of prime order q with generator g , as defined in Section 3.1. Let $sk \xleftarrow{\$} \mathbb{Z}_q$ be sampled uniformly at random, and let $pk = g^{sk}$. Then pk is uniformly distributed over $\mathbb{G}$.

Lemma 3.6.1 (Single-Sample Indistinguishability of Rejection-Sampled Keys).

Fix $m_c \in \{0, 1\}^\ell$. Let $\mathcal{D}_0$ be the distribution obtained by sampling $sk \xleftarrow{\$} \mathbb{Z}_q$ and outputting $pk = g^{sk}$. Let $\mathcal{D}_1$ be the distribution obtained by repeatedly sampling $sk \xleftarrow{\$} \mathbb{Z}_q$, setting $pk = g^{sk}$, and outputting the first pk such that $\text{PRF}_{dkey}(pk) = m_c$.

If PRF is secure, then for every PPT adversary $\mathcal{A}$, $|\Pr[\mathcal{D}_1^{\mathcal{A}} = 1] - \Pr[\mathcal{D}_0^{\mathcal{A}} = 1]| \leq \text{negl}(\lambda)$.

Proof. We use a standard hybrid argument.

Define a hybrid distribution $\mathcal{H}$ that is identical to $\mathcal{D}_1$, except that PRF_{dkey} is replaced by a truly random function $\mathcal{R} : \mathbb{G} \rightarrow \{0, 1\}^\ell$.

Since PRF is secure, any PPT adversary that distinguishes $\mathcal{D}_1$ from $\mathcal{H}$ can be used to distinguish PRF_{dkey} from a truly random function. Hence, by Definition 2.6.1, $|\Pr[\mathcal{D}_1^{\mathcal{A}} = 1] - \Pr[\mathcal{H}^{\mathcal{A}} = 1]| \leq \text{negl}(\lambda)$.

It remains to compare $\mathcal{H}$ and $\mathcal{D}_0$. In the hybrid distribution, each candidate public key is generated by sampling $sk \xleftarrow{\$} \mathbb{Z}_q$ and setting $pk = g^{sk}$. By Definition 3.6.1, pk is uniformly distributed over $\mathbb{G}$.

Since $\mathcal{R}$ is a truly random function independent of key generation, $\mathcal{R}(pk)$ is uniformly distributed over $\{0, 1\}^\ell$. The rejection condition $\mathcal{R}(pk) = m_c$ therefore holds with probability $2^{-\ell}$, independently of pk .

Every candidate pk is sampled uniformly from $\mathbb{G}$, and the acceptance probability is identical for each pk , hence the rejection step does not bias the distribution towards any subset of $\mathbb{G}$. Formally for any $x \in \mathbb{G}$,

$$\Pr[pk = x \mid \mathcal{R}(pk) = m_c] = \frac{\Pr[\mathcal{R}(x) = m_c] \cdot \Pr[pk = x]}{\Pr[\mathcal{R}(pk) = m_c]} = \frac{2^{-\ell} \cdot \frac{1}{|\mathbb{G}|}}{2^{-\ell}} = \frac{1}{|\mathbb{G}|}.$$

Thus, conditioning on this event preserves the uniform distribution of pk . Hence, the output distribution of H is identical to $\mathcal{D}_0$. Therefore, $\Pr[\mathcal{H}^{\mathcal{A}} = 1] = \Pr[\mathcal{D}_0^{\mathcal{A}} = 1]$. and combining the above inequalities yields $|\Pr[\mathcal{D}_1^{\mathcal{A}} = 1] - \Pr[\mathcal{D}_0^{\mathcal{A}} = 1]| \leq \text{negl}(\lambda)$, as claimed. $\square$

We extend Lemma 3.6.1 to prove that the indistinguishability of a single rejection-sampled key extends to multiple keys generated independently over time.

Lemma 3.6.2 (Long-term Indistinguishability of Rejection-Sampled Keys). *Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ be a secure pseudorandom function. Assume $\ell \ll \log_2(q)$. Let $\mathcal{D}_0$ be the uniform distribution over $\mathbb{G}$. Let $\mathcal{D}_1$ be the distribution obtained by independently generating t public keys, where t is $\text{poly}(\lambda)$, using single-sample rejection sampling for fixed messages $m_c^{(1)}, \dots, m_c^{(t)}$ via PRF_{dkey} .*

Any compromised adversary $\mathcal{A}$ has negligible advantage in distinguishing t keys sampled from $\mathcal{D}_1$ from t independent uniform keys from $\mathcal{D}_0$: $\left| \Pr[\mathcal{D}_1^{t, \mathcal{A}} = 1] - \Pr[\mathcal{D}_0^{t, \mathcal{A}} = 1] \right| \leq \text{negl}(\lambda)$.

Proof. We prove long-term indistinguishability by a standard hybrid argument.

Let $\mathcal{H}_0, \dots, \mathcal{H}_t$ be a sequence of hybrid experiments, where in $\mathcal{H}_i$ the first i keys are generated via rejection sampling conditioned on $\text{PRF}_{dkey}(pk) = m_c^{(i)}$, and the remaining $(t - i)$ keys are uniform.

By Lemma 3.6.1, every single rejection-sampled key is computationally indistinguishable from uniform to any compromised adversary. By the assumption $\ell \ll \log_2(q)$, the number of keys satisfying $\text{PRF}_{dkey}(pk) = m_c^{(i)}$ is sufficiently large, so rejection sampling preserves the effective uniformity of the output keys.

By hybrid composition, the distinguishing advantage between $\mathcal{H}_{i-1}$ and $\mathcal{H}_i$ is negligible for each i . Summing over t hybrids, where t is polynomial in λ , preserves negligibility. Formally: $\left| \Pr[\mathcal{D}_1^{t, \mathcal{A}} = 1] - \Pr[\mathcal{D}_0^{t, \mathcal{A}} = 1] \right| \leq \sum_{i=1}^t \text{negl}(\lambda) = \text{negl}(\lambda)$. $\square$

As we will see in Section 3.8, the assumption $\ell \ll \log_2(q)$ holds easily for values of q commonly used for Signal.

Now we use the previous lemmas to prove the indistinguishability of the standard Signal and the Covert-Keyspace scheme.

Lemma 3.6.3 (Indistinguishability of Covert-Keyspace Scheme). *Assume PRF is secure, $\ell \ll \log_2(q)$, and the number of covert messages sent under the same $dkey \in \{0, 1\}^\lambda$ is polynomial in λ . Then for any compromised PPT adversary $\mathcal{A}$,*

$$\text{DistinguishAdv}_{\mathcal{A}}^{\Sigma, \Sigma'}(\lambda) \leq \text{negl}(\lambda).$$

Proof. In Σ' , the only difference in encryption from Σ is the use of rejection-sampled ephemeral public keys to encode covert messages. All other protocol operations are identical to standard Signal. Therefore, any distinguishing advantage between the outputs of the full games $\text{DistGame}_0^{\Sigma, \mathcal{A}}$ and $\text{DistGame}_1^{\Sigma', \mathcal{A}}$ must come solely from distinguishing the ratchet keys.

By Lemma 3.6.1, each single rejection-sampled public key is computationally indistinguishable from a uniform key in G . By Lemma 3.6.2, this extends to t keys generated

independently over time. Hence, the Covert-Keyspace scheme is indistinguishable from standard Signal. $\square$

We now leverage Lemma 3.6.3 to extend standard Signal security guarantees to the Covert-Keyspace scheme.

Lemma 3.6.4 (Preservation of Signal Security Guarantees). *Let $\mathcal{P} \in \{\text{CONF}, \text{INT}, \text{FS}, \text{PCS}\}$ denote any ACCE-style security property of secure messaging, namely message confidentiality, message integrity/authenticity, forward secrecy, and post-compromise security.*

For all standard PPT adversaries $\mathcal{A}$, $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma}(\lambda) + \text{negl}(\lambda)$.

Proof. The preservation of the security guarantees reduces to the following observation: the modifications in Σ' do not introduce any distinguishing advantage beyond a negligible term.

(For completeness, the full proof is included in Appendix A). $\square$

We now show that the covert messages encoded within these keys remain confidential to any adversary that does not know $dkey$.

Lemma 3.6.5 (Confidentiality of the Covert Message). *Let PRF_{dkey} be secure and let $dkey$ be unknown to the adversary, then for any compromised PPT adversary $\mathcal{A}$, $\text{ConfidentialAdv}_{\mathcal{A}}^{\Sigma'}(\lambda) \leq \text{negl}(\lambda)$.*

Proof. The proof reduces to the pseudorandomness of the PRF. In particular, if an adversary can distinguish between covert messages, then one can construct a distinguisher that breaks the pseudorandomness of PRF_{dkey} .

(For completeness, the full proof is included in Appendix A). $\square$

Theorem 3.6.1 (Security of the Covert-Keyspace Signal Scheme). *Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ be a secure pseudorandom function. Assume $\ell \ll \log_2(q)$ and the number of covert messages sent under the same $dkey \in \{0, 1\}^\lambda$ is $\text{poly}(\lambda)$.*

Under the stated assumptions, Σ' achieves indistinguishability from Σ , preserves all ACCE-style security properties of Σ , and ensures confidentiality of covert messages.

Proof. Follows directly from Lemma 3.6.3, Lemma 3.6.4, and Lemma 3.6.5. $\square$

3.7 Repetition of Keys

The security guarantees against standard adversaries of the Signal protocol, and consequently of the covert-keyspace scheme, critically rely on the assumption that ratchet keys are never repeated. If a ratchet key were ever reused across distinct ratchet epochs, it could lead to two types of vulnerabilities in our scheme:

1. **Breakage of forward secrecy and post-compromise security.** The asymmetric ratchet relies on fresh ephemeral key pairs to inject new entropy into the root key. If a previously used ratchet key is repeated, an adversary who previously observed or compromised the key could recompute future root keys and message keys, violating post-compromise security (Perrin et al., 2025).
2. **Potential leakage of covert messages.** In the covert-keyspace scheme, covert messages are embedded via rejection-sampled public keys. If a public key is reused, the same covert message would be transmitted multiple times. An adversary observing multiple transmissions of the same key could infer that the same covert message is being sent, thereby reducing the channel's effective secrecy.

Next, we justify why repetition of ratchet keys is computationally negligible in practice.

Lemma 3.7.1 (Ratchet Key Collision Bound). *Let $(pk_1, \dots, pk_n) \in \mathbb{G}$ be ratchet public keys generated within a single session of the covert-keyspace scheme over a group $\mathbb{G}$ of prime order q . Then the probability that at least one key is repeated satisfies*

$$\Pr [\exists i \neq j : pk_i = pk_j] \leq \binom{n}{2} q^{-1} + \text{negl}(\lambda).$$

Proof. A key repetition occurs if and only if there exists a pair of distinct indices $i \neq j$ such that $pk_i = pk_j$ for $1 \leq i < j \leq n$.

In the covert-keyspace scheme, public keys are generated via rejection sampling subject to the constraint $\text{PRF}_{dkey}(pk) = m_c$. By Lemma 3.6.3, the resulting distribution of accepted keys is computationally indistinguishable from the uniform distribution over $\mathbb{G}$. We therefore consider a hybrid experiment $\mathcal{H}$ in which each pk_i is sampled independently and uniformly at random from $\mathbb{G}$. By Lemma 3.6.3, any PPT distinguisher can distinguish the real scheme from $\mathcal{H}$ with at most negligible probability $\text{negl}(\lambda)$.

Conditioned on $\mathcal{H}$, for any fixed pair (i, j) with $i \neq j$, the probability that $pk_i = pk_j$ is exactly q^{-1} , since both values are independent uniform samples from $\mathbb{G}$.

Applying the union bound over all unordered pairs (i, j) yields

$$\Pr [\exists i \neq j : pk_i = pk_j] \leq \binom{n}{2} q^{-1} + \text{negl}(\lambda). \quad \square$$

For Curve25519, which is the primary elliptic curve used by Signal (Perrin et al., 2025), $q \approx 2^{252}$. Therefore, if $n = 2^{30}$ keys, which far exceeds realistic usage, the collision probability is $\Pr[\text{repetition}] \leq \frac{(2^{30})^2}{2 \cdot 2^{252}} = \frac{2^{60}}{2^{253}} = 2^{-193}$. Consequently, the likelihood of generating a repeated ratchet key is negligible in both Signal and the covert-keyspace scheme.

3.7.1 Rekeying Policy

The Lemma 3.6.2 ensures that up to t rejection-sampled keys are computationally indistinguishable from uniform, where t is $\text{poly}(\lambda)$. However, the pseudorandom key $dkey$ must be rotated in the long term. In particular, if $dkey$ has length λ bits, it should

be replaced after roughly $2^{\lambda/2}$ keys to avoid potential birthday-style attacks on the PRF outputs.

3.8 Computational Feasibility

Although we have shown that covert messages of length ℓ can be securely embedded, the feasibility of the construction is constrained by the cost of rejection sampling. As previously mentioned, each candidate public key matches a chosen ℓ -bit covert message with probability $2^{-\ell}$, so successful embedding requires an expected 2^ℓ candidate generations.

For larger values of ℓ , this exponential cost rapidly becomes prohibitive. For example, when $\ell = 20$, the expected number of required samples exceeds one million, significantly increasing message generation latency and potentially introducing observable timing anomalies or resource usage patterns. In contrast, restricting the covert message length to $\ell \leq 16$ bounds the expected rejection sampling cost to at most 65,536 candidate public keys. This level of computation is feasible on modern hardware; implementations of Curve25519 scalar multiplication can perform tens of thousands of operations per second (Sasdrich et al., 2017).

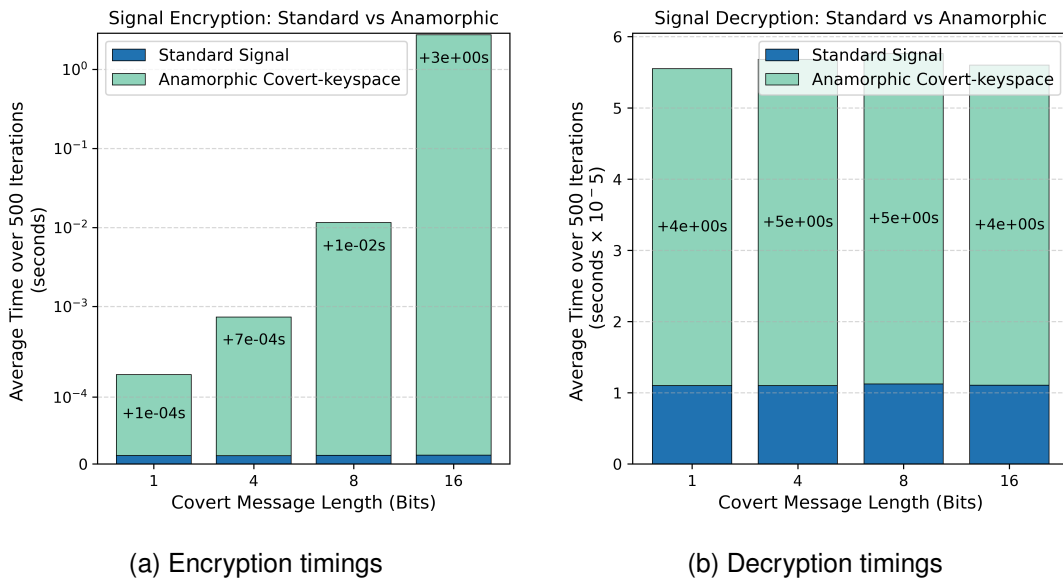


Figure 3.2: Timing overhead introduced by the covert-keyspace scheme compared to standard Signal across varying covert message sizes and a fixed 256-byte message. The labels shown within each bar represent the raw added time; percentage increases were too large to provide insight. Measured on the Eddie HPC cluster. Implementations were CPU-only, using Python *cryptography* with X25519 for key exchange, AES-GCM (256-bit key) for symmetric encryption, and HMAC as a PRF; timings via *time.perf_counter()*, averaged over 500 runs.

The results in Figure 3.2 confirm the expected theoretical behaviour. Encryption time increases rapidly as the length of the covert message grows, reflecting the exponential

cost of rejection sampling. The average encryption time increases from approximately 1.37×10^{-4} seconds when embedding a 1-bit covert message, to 2.74 seconds for 16-bits. By comparison, encryption for 20-bit covert messages took over 42 seconds on average. This behaviour closely matches the theoretical expectation that the number of required samples grows proportionally to 2^ℓ . Limiting covert messages to 16 bits therefore provides a practical balance between covert bandwidth and computational feasibility.

In contrast, the cost of decryption remains essentially constant regardless of the length of the covert message due to the lack of rejection sampling. As a result, decryption times remain close to 5.6×10^{-5} seconds across all tested covert message lengths, only slightly higher than the 1.1×10^{-5} seconds observed for the standard protocol.

This trade-off between covert bandwidth and computational feasibility is inherent to rejection sampling-based steganographic constructions and has been observed in prior work on steganography and subliminal channels (von Ahn and Hopper, 2004; Horel et al., 2019).

In Appendix B, we present an alternate construction which trades Alice’s exponential runtime for a one-time exponential pre-computation performed by Bob. The remainder of this paper refers only to the covert-keyspace scheme.

3.9 Robustness

We refine our definition of robustness for the covert-keyspace setting as follows.

The covert-keyspace scheme is robust if, for any PPT adversary controlling the communication channel, the intended receiver can reliably determine whether a transmitted ratchet public key encodes a covert message, and detect any modification or forgery of the covert content. Once again, we do not require the guaranteed delivery of covert messages against a compromised adversary capable of dropping messages. As we discuss later, the adversary’s ability to fail to deliver a message is more limited in this scheme.

In the covert-IV scheme, each covert message can encode a significant number of bits, enabling us to implement an authenticated payload structure. This idea is not feasible on a covert channel limited to 16 bits per message, as the MAC tag would be unacceptably short, resulting in a non-negligible forgery probability. Instead, we spread the burden of robustness across a sequence of messages.

3.9.1 Incremental Authentication

To achieve robustness under this constraint, we incrementally authenticate covert transmissions. Rather than authenticating each broadcasted covert message, we divide the transmission into fixed-size blocks, compute a MAC for each block, and distribute both the data and the authentication material across multiple ratchet steps. Although authentication material is distributed across multiple ratchet steps, verification is performed on

the reconstructed block, so any modification to even a single fragment causes the final MAC check to fail.

Control and Key Derivation

Each of these blocks will be transmitted across a series of asymmetric ratchets, where each m_c contains only a fragment of the data and authentication code. To control and coordinate this process, before each block the sender sends a covert message containing a pre-agreed unique start symbol S . Similarly, to indicate the end of the block another unique marker E is transmitted. This allows the receiver to reliably determine when covert data is present; once the S marker is detected, subsequent ratchet public keys are interpreted as carrying covert payload fragments until the end marker E is encountered.

To create an authentication code, we derive subkeys from $dkey$. Similarly to Section 2.8: $(dkey_{prf}, dkey_{mac}) := \text{KDF}(dkey)$, where $dkey_{prf}$ is used for rejection sampling public keys and $dkey_{mac}$ serves as the base authentication key.

For block-level key separation, we derive a per-block MAC key from the base authentication key and the root key rk in use before the covert message containing S is transmitted. This ensures that the per-block key is unique, as rk should not repeat by design in Signal. Usefully, both the sender and the recipient share the same rk (Perrin et al., 2025). Therefore, for block j : $k_{mac}^{(j)} := \text{KDF}(dkey_{mac} \parallel rk)$.

Block Partitioning

Let $M \in \{0, 1\}^n$ denote the covert transmission Alice wishes to send. We pad M to a multiple of 128 bits and write $M = B_1 \parallel B_2 \parallel \dots \parallel B_s$, where each block $B_j \in \{0, 1\}^{128}$. Note that, for the following demonstration, we use blocks of 128-bits; this may be adjusted based on the needs of the communicating parties.

For each block j , a 128-bit MAC $\tau_j := \text{MAC}_{k_{mac}^{(j)}}(B_j)$ is computed. The block and tag are split into sixteen 8-bit fragments $B_j = p_{j,1} \parallel \dots \parallel p_{j,16}$, $\tau_j = t_{j,1} \parallel \dots \parallel t_{j,16}$, where $p_{j,i}, t_{j,i} \in \{0, 1\}^8$. For each i , the covert payload is $m_c^{(j,i)} = p_{j,i} \parallel t_{j,i}$.

Full Procedure

To transmit block B_j , Alice begins by sending a covert message containing S to signal the beginning of a covert block transmission. For each of $i = 1, \dots, 16$, she generates a ratchet public key $pk^{(j,i)}$ via rejection sampling such that $\text{PRF}_{dkey_{prf}}(pk^{(j,i)}) = m_c^{(j,i)}$. She then transmits the protocol message containing $pk^{(j,i)}$. After all sixteen such messages have been sent, Alice transmits the end marker E to signal completion of the block.

Upon detecting the start marker S , Bob collects all the following ratchet public keys $pk^{(j,1)}, \dots, pk^{(j,16)}$ until receiving E . For each public key he receives, the covert payload is extracted by computing $m_c^{(j,i)} = \text{PRF}_{dkey_{prf}}(pk^{(j,i)})$. He then parses each value as $m_c^{(j,i)} = p_{j,i} \parallel t_{j,i}$. Bob reconstructs the candidate block and tag as $\hat{B}_j =$

$p_{j,1} \parallel \dots \parallel p_{j,16}$, $\hat{\tau}_j = t_{j,1} \parallel \dots \parallel t_{j,16}$. He derives the corresponding block authentication key $k_{mac}^{(j)}$ and verifies that $\hat{\tau}_j = \text{MAC}_{k_{mac}^{(j)}}(\hat{B}_j)$. If verification succeeds, Bob accepts B_j as authentic. Otherwise, he rejects all the messages in the block.

Definition 3.9.1 (Robustness of Σ'). Let $\mathcal{A}$ be any compromised PPT adversary interacting with the scheme. Define the robustness advantage as $\text{RobustAdv}_{\mathcal{A}}^{\Sigma'} := \Pr[\text{AcceptInvalid}]$, where AcceptInvalid is the event that the receiver accepts a covert message block with a tag τ that was not honestly generated using the corresponding per-block key $k_{mac}^{(j)}$. We say that Σ' is robust if for all PPT adversaries $\mathcal{A}$, $\text{RobustAdv}_{\mathcal{A}}^{\Sigma'} \leq \epsilon$, for small ϵ determined by the length of τ .

Lemma 3.9.1 (Robustness of Incrementally Authenticated Covert Blocks). *Let Σ' be the covert-keyspace scheme using per-block MACs, where each covert block B_j is split into sixteen 8-bit fragments $p_{j,1}, \dots, p_{j,16}$, and authenticated with a 128-bit MAC τ_j under a per-block key $k_{mac}^{(j)}$ unknown to the adversary. Then for any compromised PPT adversary $\mathcal{A}$ making at most q attempts, the robustness advantage satisfies $\text{RobustAdv}_{\mathcal{A}}^{\Sigma'} \leq q \cdot 2^{-128}$, where q is the number of adversarial attempts.*

Proof. Each block B_j is authenticated under a unique key $k_{mac}^{(j)}$ and reconstructed from sixteen ratchet steps. For any block not honestly generated using $k_{mac}^{(j)}$, the reconstructed pair $(\hat{B}_j, \hat{\tau}_j)$ is independent of the MAC key. By EUF-CMA, the probability that a dishonest pair verifies $(\hat{\tau}_j = \text{MAC}_{k_{mac}^{(j)}}(\hat{B}_j))$ is at most 2^{-128} for any single attempt. Taking a union bound over at most q adversarial attempts gives $\Pr[\text{AcceptInvalid}] \leq q \cdot 2^{-128}$. This shows that the probability of accepting any invalid covert block is negligible and bounded by the MAC length and the number of adversarial attempts. $\square$

This bound captures both adversarial forgery and accidental acceptance. An adversary attempting to construct a valid block must correctly forge all sixteen 8-bit fragments and produce a valid MAC under the unknown per-block key, which succeeds with probability at most 2^{-128} . On the other hand, when no covert block is transmitted, the reconstructed fragments are uniformly random, and the probability that they collectively form a valid block passing MAC verification is also at most 2^{-128} . Thus, whether arising from adversarial action or random chance, any invalid block is accepted only with negligible probability per attempt.

Path Towards Reliability

While the scheme guarantees robustness, the sender has no way to know whether the covert transmission M_c was successfully received. One possible extension is to have the receiver compute a Merkle tree over the recovered fragments $\{p_{j,i}\}_{i=1}^{16}$ using an agreed hash function. The recipient could then return the Merkle root to the sender over a covert or overt channel.

Upon receiving the root, the sender can verify consistency with their locally computed root. This provides explicit confirmation that all fragments were received and assembled correctly. There is potential for further extension where Merkle authentication paths

could be used to selectively retransmit only missing fragments rather than the entire block.

3.10 Attacks

Many potential attacks on this covert channel are analogous to those against the covert-IV scheme; fundamentally, both schemes expose an unencrypted public value that appears random while concealing a covert message. We focus on motivating how the covert-keyspace scheme provides security against compromised adversaries and interesting features of the underlying Signal algorithm that differ from AES-GCM.

3.10.1 Safety under Compromised Assumptions

The compromised-sender assumption allows the adversary a degree of control of the encryption, however, they have no control over the creation of the asymmetric ratchet keys. When the public key is later observed or requested, it is indistinguishable from a regular ratchet key.

The compromised-receiver assumption gives the adversary the power to intercept and decrypt all communications and Diffie-Hellman exchanges. The adversary can do this by requesting any relevant secret key belonging to the receiver. However, because the existence of $dkey$ is only known to the communicating parties, nothing about the covert message is revealed.

3.10.2 Potential Attacks

- **Replay Attack.** The robust scheme uses a different MAC key for each block, preventing successful replay across different blocks. Replaying within the same block would cause the receiver to compute a different authentication tag and discard the entire block.
- **Re-Encryption.** Similarly to the covert-IV scheme, a person-in-the-middle attack is possible due to compromised assumptions, whereby the adversary is communicating separately with Alice and Bob and forwarding messages. This attack does not compromise the scheme's robustness, as the adversary would still need to forge a block successfully. However, it removes the covert channel as the adversary can effectively control the encryption randomness, bypassing the restriction imposed by the compromised-sender assumption.
- **Message Dropping.** Incremental authentication is especially weak against message dropping attacks; if any fragment of the block is dropped, Bob will fail to authenticate the block and ignore all content. However, the Signal protocol mitigates this weakness. If the adversary chooses not to deliver a communication containing an asymmetric ratchet public key, the communication between the parties breaks. Alice will compute the new root and chaining keys, while Bob has no knowledge of this change. This attack is detectable when the next communication is attempted. Additionally, if the channel of communication is destroyed, it

cannot be monitored.

- **Timing Attacks.** We acknowledge that as the number of covert bits per message rises, the computation required to successfully rejection sample a public key becomes increasingly detectable. Our results in Figure 3.2 highlight a significant increase in encryption times when we adopt the covert-keyspace for 16-bit covert messages. We suggest that the observable effects could be reduced using the extended covert-keyspace construction in Appendix B.

3.11 Discussion

The Signal-based construction provides a covert channel embedded within the normal operation of the Double Ratchet protocol. Although the bandwidth per message is lower than in the covert-IV construction, the scheme still supports a sustainable covert channel in the long term and allows parties to initiate covert communication at any time after establishing the double key. However, unlike the AES-GCM scheme, where covert communication relies on manipulating IV generation, the Signal protocol already transmits fresh public ratchet keys as part of each ratchet step. Consequently, the covert channel does not depend on assumptions about IV randomness or implementation choices; it arises naturally from the protocol’s structure. This makes the covert-keyspace scheme harder to prevent in systems that use Signal. More generally, the same approach could be extended to other protocols that frequently generate and transmit ephemeral public keys.

As in the covert-IV construction, this scheme also allows covert communication between parties who are not directly communicating with one another. Suppose Alice and Charlie both maintain ordinary Signal conversations with Bob and can observe the network traffic exchanged with him. The ratchet public keys are transmitted in the clear as part of the protocol, therefore, a third party possessing the shared double key $dkey$ can derive the covert payload from these public values without knowledge of the underlying session keys. This allows Alice and Charlie to exchange covert information indirectly through their communications with Bob, while Bob himself need not be aware that any covert channel exists.

Finally, the scheme provides robustness without requiring an external synchronisation mechanism. As discussed in Section 1.3, many anamorphic constructions, including the covert-IV scheme, rely on shared counters or additional coordination between communicating parties. In contrast, our scheme leverages the Double Ratchet protocol’s existing synchronisation guarantees. The use of markers S and E allows the receiver to identify and delimit covert transmissions directly from observed ratchet public keys, without maintaining any additional shared state beyond that already required by the protocol. This design reduces the risk of desynchronisation and improves tolerance to moderate message loss or reordering.

Chapter 4

Conclusions

A limitation of prior work on anamorphic encryption is its exclusive focus on PKE schemes. In this dissertation, we have introduced and analysed constructions that extend the anamorphic paradigm to other widely used cryptographic structures, symmetric encryption and Signal. These first attempts leverage a double key to exploit protocol randomness, and we anticipate that similar techniques could be applied to other similar randomness channels in probabilistic encryption algorithms. We leave this as a promising direction for future research.

Several open problems remain. First, like prior anamorphic schemes, our constructions are inherently vulnerable to re-encryption attacks. An adversary who possesses the underlying encryption keys can decrypt, manipulate, and re-encrypt ciphertexts, effectively overwriting the covert channel. While re-encryption attacks could remove covert channels for targeted individuals, in large-scale deployments of general-purpose encryption the sheer volume of encrypted traffic dilutes the likelihood that a single covert message will be singled out, making such attacks less practical or concerning.

Second, while our methods could be adapted to exploit sources of randomness in existing protocols, such channels must provide sufficient entropy. Therefore, they cannot be deterministically derived from the shared secret due to the compromised assumptions. Ironically, Kerckhoffs' principle, which asserts that security should not rely on obscurity but only on secret keys (Kerckhoffs, 1883), often prevents us from securing a channel against a compromised adversary.

Finally, a fundamental limitation shared by all practical schemes under our threat model is susceptibility to denial-of-service attacks. Since the compromised adversary controls the communication channel, they can trivially block, delay, or discard messages, preventing any overt or covert communication from reaching the intended recipient. Although such attacks cannot be mitigated cryptographically, complete communication denial also inherently precludes meaningful surveillance.

Bibliography

- Apple Inc. Apple can no longer offer advanced data protection in the uk, 2025. URL <https://support.apple.com/en-us/122234>.
- Fabio Banfi, Konstantin Gegier, Martin Hirt, Ueli Maurer, and Guilherme Rito. Anamorphic encryption, revisited. In *Lecture Notes in Computer Science*, pages 3–32. Springer, 2024.
- Mihir Bellare, Joe Kilian, and Phillip Rogaway. The security of cipher block chaining. In *Lecture Notes in Computer Science*, pages 341–358. Springer, 1994.
- Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In *Advances in Cryptology – CRYPTO ’96*, volume 1109 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 1996.
- David Brumley and Dan Boneh. Remote timing attacks are practical. In *Proceedings of the 12th USENIX Security Symposium*, pages 1–14. USENIX Association, 2003.
- Davide Carnemolla, Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic resistant encryption: the good, the bad and the ugly. In *Lecture Notes in Computer Science*, pages 472–503. Springer Nature, 2025. URL <https://eprint.iacr.org/2025/233>. *Lecture Notes in Computer Science*.
- Dario Catalano, Antonio Giunta, and Marco Migliaro. Extending anamorphic encryption to hybrid encryption. *IACR Cryptology ePrint Archive*, 2024.
- Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the signal messaging protocol. Technical report, *Cryptology ePrint Archive*, 2019. URL <https://eprint.iacr.org/2016/1013.pdf>.
- Whitfield Diffie and Martin Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, November 1976. doi: 10.1109/TIT.1976.1055638.
- Yevgeniy Dodis and Eli Goldin. Anamorphic-resistant encryption; or why the encryption debate is still alive. *Cryptology ePrint Archive*, Paper 2025/293, 2025. URL <https://eprint.iacr.org/2025/293>.
- Danny Dolev and Andrew C. Yao. On the security of public key protocols. In *Proceedings of FOCS*, pages 350–357, 1981.

- Morris J. Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. Technical Report NIST SP 800-38D, National Institute of Standards and Technology, 2007. URL <https://nvlpubs.nist.gov>.
- Edinburgh Compute and Data Facility. Edinburgh compute and data facility web site. <https://www.ecdf.ed.ac.uk>, December 2021. University of Edinburgh, accessed 1 December 2021.
- Freedom House. Freedom on the net 2021: China country report, 2021. URL <https://freedomhouse.org>.
- Thibaut Horel, Sunoo Park, Silas Richelson, and Vinod Vaikuntanathan. How to subvert backdoored encryption: Security against adversaries that decrypt all ciphertexts, 2019. URL <https://eprint.iacr.org/2018/748>.
- Panos Kampanakis, Matt Campagna, Eric Crockett, Adam Petcher, and Shay Gueron. Practical challenges with aes-gcm and the need for a new cipher, 2024. URL <https://csrc.nist.gov>.
- Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3 edition, 2014.
- Stephen Kent and Karen Seo. Security architecture for the internet protocol. Technical Report RFC 4301, Internet Engineering Task Force, 2005. URL <https://datatracker.ietf.org/doc/html/rfc4301>.
- Auguste Kerckhoffs. La cryptographie militaire. *Journal des Sciences Militaires*, pages 5–38, 161–191, 1883. English translation available online.
- Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In *Advances in Cryptology — CRYPTO '96*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
- Andreas Lichter, Max Löffler, and Sebastian Sieglöcher. The long-term costs of government surveillance. *Journal of the European Economic Association*, 19(2):741–779, 2021.
- Melanie Lidman. Activists say iran has aired at least 97 coerced confessions, 2026. URL <https://abcnews.go.com>.
- David A. McGrew and John Viega. The galois/counter mode of operation (gcm), 2007. URL <https://csrc.nist.gov>.
- Trevor Perrin, Moxie Marlinspike, and Rolfe Schmidt. *The Double Ratchet Algorithm*. Signal Messenger LLC, 2025. URL <https://signal.org/docs/specifications/doubleratchet/>. Revision 4.
- Giuseppe Persiano, Hieu Duong, Hoang Phan, and Moti Yung. Anamorphic encryption: Private communication against a dictator. Technical Report Report 2022/639, Cryptology ePrint Archive, 2022. URL <https://eprint.iacr.org/2022/639>.
- Alina Polyakova and Chris Meserole. Exporting digital authoritarianism: The russian and chinese models, 2019. URL <https://www.brookings.edu>.

- Eric Rescorla. The transport layer security (tls) protocol version 1.3. Technical Report RFC 8446, Internet Engineering Task Force, 2018. URL <https://datatracker.ietf.org/doc/html/rfc8446>.
- Sasdrich, Güneysu, et al. Low-latency x25519 hardware implementation: breaking the 100 microseconds barrier. *ScienceDirect*, 2017.
- Tom Van Goethem and Wouter Joosen. Timeless timing attacks: Exploiting concurrency to leak secrets over remote connections. In *Proceedings of the 29th USENIX Security Symposium*, pages 1985–2002. USENIX Association, 2020.
- John Villasenor. Recording everything: Digital storage as an enabler of authoritarian governments. Technical report, Brookings Institution, 2011.
- Luis von Ahn and Nicholas J. Hopper. Public-key steganography. In *EUROCRYPT*, 2004.
- Wikipedia Contributors. Rejection sampling, 2004. URL https://en.wikipedia.org/wiki/Rejection_sampling.
- Tatu Ylönen and Chris Lonvick. The secure shell (ssh) protocol architecture. Technical Report RFC 4251, Internet Engineering Task Force, 2006. URL <https://datatracker.ietf.org/doc/html/rfc4251>.

Appendix A

Complete Proofs

A.1 Preservation of Signal Guarantees

Lemma A.1.1 (Preservation of Signal Security Guarantees). *Let $\mathcal{P} \in \{\text{CONF}, \text{INT}, \text{FS}, \text{PCS}\}$ denote any ACCE-style security property of secure messaging, namely message confidentiality, message integrity/authenticity, forward secrecy, and post-compromise security.*

For all standard PPT adversaries $\mathcal{A}$, $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma}(\lambda) + \text{negl}(\lambda)$.

Proof. The only difference in encryption between Σ' and standard Signal Σ is the use of rejection-sampled ephemeral keys to encode covert messages. By Lemma 3.6.3, these keys are computationally indistinguishable from uniform keys to any compromised PPT adversary.

Hence, any standard adversary $\mathcal{A}$ attacking a property $\mathcal{P}$ in Σ' can be simulated by an adversary $\mathcal{A}$ interacting with Σ , with at most a negligible loss in advantage due to key indistinguishability. Formally, $\text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma'}(\lambda) \leq \text{SecurityAdv}_{\mathcal{A}}^{\mathcal{P}, \Sigma}(\lambda) + \text{negl}(\lambda)$. $\square$

A.2 Covert-Keyspace Confidentiality

Lemma A.2.1 (Confidentiality of the Covert Message). *Let PRF_{dkey} be secure and let $dkey$ be unknown to the adversary, then for any compromised PPT adversary $\mathcal{A}$,*

$$\text{ConfidentialAdv}_{\mathcal{A}}^{\Sigma'}(\lambda) \leq \text{negl}(\lambda).$$

Proof. We prove this via a hybrid argument, similar to Lemma 3.6.1.

We define a hybrid distribution $\mathcal{H}$ where the pseudorandom function PRF_{dkey} used in rejection sampling is replaced with a truly random function $\mathcal{R} : G \rightarrow \{0, 1\}^\ell$. Each candidate public key pk is then uniform over G by Definition 3.6.1, and independent

of the covert message $m_c^{(b)}$. The rejection sampling step simply selects the first pk satisfying $\mathcal{R}(pk) = m_c^{(b)}$, which does not leak information about b .

Observe that in $\mathcal{H}$, the output distribution of pk is statistically identical for $b = 0$ and $b = 1$, because $\mathcal{R}(pk)$ is independent of b . Therefore, any adversary $\mathcal{A}$ cannot distinguish which covert message was encoded.

Finally, any non-negligible advantage of $\mathcal{A}$ in the real game Σ' would imply a non-negligible advantage in distinguishing PRF_{dkey} from a truly random function. By Definition 2.6.1, this advantage is negligible.

Hence,

$$\text{ConfidentialAdv}_{\mathcal{A}}^{\Sigma'}(\lambda) \leq \text{negl}(\lambda),$$

proving that the covert message remains confidential against any adversary without $dkey$. $\square$

Appendix B

Masked Covert-Keyspace Construction

We now present an alternate construction that embeds a small covert message into the asymmetric ratchet while moving the computational cost from the sender to the receiver. We refer to this construction as the masked covert-keyspace scheme.

Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order q used by the protocol. Let $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \mathbb{Z}_q$ be a secure keyed pseudorandom function. Analogous to the covert-IV scheme, we initialise a counter ctr which increments with each asymmetric ratchet.

For a covert message $m_c \in \{0, 1\}^\ell$

Alice (encoding)

1. Compute $r_i := \text{PRF}_{dkey}(ctr) \pmod{q}$.
2. Form the ephemeral private key $sk_i \leftarrow r_i + m_c \pmod{q}$.
3. Compute the corresponding public key $pk_i \leftarrow g^{sk_i}$.

Bob (decoding)

1. Offline, iterate through the covert message space $\mathcal{M} = \{0, \dots, 2^\ell - 1\}$ and pre-compute the table $\mathcal{T} = \{(m_c, g^{m_c}) : m_c \in \mathcal{M}\}$.
2. Upon receipt of a message containing pk_i and with known ctr , compute $r_i = \text{PRF}_{dkey}(ctr)$ and $R_i \leftarrow g^{r_i}$.
3. Compute $X \leftarrow pk_i \cdot R_i^{-1} = g^{m_c}$.
4. Lookup m_c by finding $m_c \in \mathcal{M}$ with $g^{m_c} = X$ via $\mathcal{T}$.

B.1 Comparison with Rejection Sampling

The classical rejection sampling approach requires Alice to generate an expected 2^ℓ candidate keys until one satisfies the PRF predicate. In contrast, the masked-DH construction requires:

- Alice: a single canonical key generation per covert message (constant cost).
- Bob: an offline precomputation of 2^ℓ group elements and constant-time online decoding per message (table lookup).

Thus, the scheme trades Alice's exponential runtime for Bob's one-time exponential precomputation. This trade is appropriate in settings where the sender may be coerced or resource-constrained, while the receiver can amortise offline computation or maintain a small lookup table.

B.2 Security of the Masked Covert-Keyspace Scheme

We now provide the intuition behind why the masked covert-keyspace meets the same high-level security goals as the rejection sampling covert-keyspace scheme: (i) the masked public keys are computationally indistinguishable from ordinary ephemeral keys to a compromised adversary that does not know $dkey$; (ii) the covert message m_c remains confidential to such adversaries; and (iii) the usual security guarantees of the underlying Signal protocol are preserved for standard adversaries.

We use the same security model as for the covert-keyspace scheme and make the same assumption that ratchet keys are not reused under the same long-term key, and additionally that ctr does not repeat under the same $dkey$.

Indistinguishability of Masked Keys

Each masked private key is computed as

$$sk_i = r_i + m_c \pmod{q}, \quad r_i := \text{PRF}_{dkey}(ctr).$$

For any fixed m_c , adding it to the pseudorandom scalar r_i preserves indistinguishability from uniform in $\mathbb{Z}_q$. Formally, if a PPT adversary $\mathcal{A}$ could distinguish g^{sk_i} from a standard ephemeral public key g^{sk} , one can construct a PRF distinguisher $\mathcal{B}$ that uses $\mathcal{A}$ to distinguish PRF_{dkey} from a truly random function. Therefore, the masked public keys are computationally indistinguishable from ordinary ephemeral keys.

Preservation of Signal Security

Since the only modification in the protocol is the masked ratchet key generation, and this distribution is indistinguishable from uniform, any attack on standard Signal security properties (message confidentiality, forward secrecy, post-compromise security) against the masked covert-keyspace scheme can be reduced to the same attack against standard Signal. By the reductions in Lemma 3.6.4, the security guarantees of Signal are preserved up to negligible differences.

Confidentiality of the Covert Message.

The covert message m_c is recovered by Bob using knowledge of $dkey$ and precomputed tables. A compromised adversary without $dkey$ cannot distinguish which m_c was

embedded: replacing PRF_{dkey} with a truly random function produces uniform r_i . The distributions $r_i + m_c^{(0)}$ and $r_i + m_c^{(1)}$ are identical, since adding a fixed value (the covert message) to a uniformly random element shifts the distribution modulo q , which preserves uniformity.

Thus, any compromised PPT adversary without access to $dkey$ has only a negligible advantage in distinguishing which covert message was transmitted, ensuring the confidentiality of m_c .